Digitale kaart en simulatie

Onderbouwing, softwareontwerp, MQTT-contract, acceptatiecriteria en testplan

Van losse proefritten naar een algemene shuttle

DP19 - Parcours volgen

Ontwerp 1.0 - 3 november 2026

Status: ontwerp gereed voor review; geen fysieke vrijgave

Wat dit document oplevert

Een volledige foto-afgeleide topologie, één voorgestelde softwarearchitectuur, 42 klassen en 6 enums in UML, 24 meetbare acceptatiecriteria en 48 uitgewerkte testscenario's.

De digitale kaart is een model van het hele parcours, inclusief Treindepot, Reuzenrad, Arcadehal, Wildwaterbaan, Vrijevaltoren en Achtbaan. Routes worden berekend uit bestemmingen en aankomstrichting, en daarna uitgevoerd door dezelfde kern in simulatie en op de ESP32.

Belangrijke grens aan exactheid

Een foto toont verbindingen, maar levert geen gemeten baanlengtes, remweg, sensorhoogte, motorrespons of uniek herkenbare aankomstpunten. Die waarden worden niet verzonnen. Het ontwerp blokkeert hardwarevrijgave zolang noodzakelijke fysieke gegevens ontbreken.

Opgesteld op basis van de aangeleverde foto, het gesprek en de huidige main.py. De terugweg op de echte robot is nog niet aantoonbaar betrouwbaar. Eerdere simulatietests gelden daarom niet als bewijs van fysiek correct rijden.

Leeswijzer en inhoud

Gebruik de PDF-bladwijzers voor navigatie. Een grote overzichtstekening en leesbare UML-detailbladen staan achterin.

01 Waarom een digitale simulatie? 3
02 Scope, feiten en open keuzes 4
02.1 Functionele en technische eisen 5
02.2 Functionele en technische eisen 6
03 Bronfoto van het parcours 7
04 Digitale topologie en vaste IDs 8
04.1 Catalogus van alle punten 9
04.2 Catalogus van alle verbindingen 10
04.3 Servicepunten en meetplan 11
05 Routeplanning met aankomstrichting 12
06 Architectuur en verantwoordelijkheden 13
07.1 Voorgestelde mappen en bestanden 14
07.2 Voorgestelde mappen en bestanden 15
08.1 Volledig klassenregister 16
08.2 Volledig klassenregister 17
08.3 Volledig klassenregister 18
09 Toestanden en uitvoeringsinvarianten 19
10 Timing, sensoren en motoren 20
11 MQTT-topics en bevestigingen 21
11.1 Opdracht- en plancontract 22
11.2 Events, fouten en eindevidence 23
12 Nieuwe instructies tijdens een rit 24
13 Simulatiemodi en kalibratiegrenzen 25
14 Bediening, API en journaling 26
15.1 Meetbare acceptatiecriteria 27
15.2 Meetbare acceptatiecriteria 28
15.3 Meetbare acceptatiecriteria 29
16 Teststrategie en orakels 30
16.1 Testscenario's 01-08 31
16.2 Testscenario's 09-16 32
16.3 Testscenario's 17-24 33
16.4 Testscenario's 25-32 34
16.5 Testscenario's 33-40 35
16.6 Testscenario's 41-48 36
17 Traceerbaarheid 37
18 Implementatie- en migratieplan 38
19 Risico's en beheersing 39
20 Ontwerpbeslissingen en alternatieven 40
21 Aanvullingen op de oorspronkelijke vraag 41
22 Uitgevoerde ontwerpcontroles 42
23 Bronnen en begrippen 43
Bijlage A - Volledig klassendiagram 44
A.1a Kaart, stations en missie 45
A.1b Stappen, pose en berichten 46
A.2 Besturingskern 47
A.3 Interfaces en enums 48
A.4 Adapters en simulatie 49
A.5 Applicatie en browser 50
Bijlage B.1 - Volledige relatietabel 51
Bijlage B.2 - Volledige relatietabel 52
Bijlage B.3 - Volledige relatietabel 53

01 Waarom een digitale simulatie?

Probleem, doel en onderbouwing

Huidige situatie

De bestaande robotcode combineert lijnvolgen, kruispuntherkenning, afslagkeuzes, verblijftijd en terugweg in één ritprogramma. Nieuwe bestemmingen hebben daardoor herhaaldelijk nieuwe afslaglijstjes en fysieke aannames opgeleverd. De robot kan een punt missen, dezelfde lijn opnieuw accepteren of na een gewijzigde vertrekafslag met een onjuiste terugroute beginnen.

De praktijk liet ook een verkeerd niet-getelde eerste rechtsafslag zien, een dubbele vrijgavewachttijd, stops tijdens meten en een depotcriterium dat hoofdzakelijk op breedtijd leunde. Een volle accu verandert de afgelegde afstand tijdens vaste milliseconden. Deze fouten horen niet allemaal bij PID; planning, waarneming, tijdgedrag en lokalisatie moeten afzonderlijk controleerbaar zijn.

Beoogd resultaat

- Bestemmingen kiezen zonder de firmware voor iedere rit te wijzigen.
- Eén gecontroleerde kaart als bron voor planner, simulatie, bediening en robot.
- De betekenis van links/rechts afleiden uit inkomende en uitgaande verbindingen.
- Een ontvangen opdracht onderscheiden van een werkelijk uitgevoerde missie.
- Nieuwe opdrachten tijdens rijden ontvangen en op een afgesproken grens toepassen.
- Fouten reproduceerbaar onderzoeken met dezelfde input, versies en simulatietijd.

Waarom eerst een topologische simulatie?

Een topologische simulatie maakt verkeerde routeverbindingen, afslagoriëntatie, bezoekvolgorde en terugwegen zichtbaar. Dat is goedkoper en herhaalbaarder dan iedere combinatie fysiek rijden. Ruwe sensorreplay vormt daarna de brug naar de werkelijke robot. Een uitgebreid 3D- of fysicapakket is geen voorwaarde voor deze eerste doelen.

Afbakening

Een perfect routepijltje bewijst geen correcte kruispuntdetectie. Een simulator mag de ware kaartpositie niet ongemerkt aan de besturing geven; anders verdwijnt precies de fout die op de echte baan optreedt.

02 Scope, feiten en open keuzes

Bewijsstatus en prioriteiten

Status	Wat vaststaat / nog ontbreekt
Uit foto afgeleid	Zwarte lijnstructuur, vijf attractietakken, depot, zes lussen; topologische catalogus in hoofdstuk 04.
Uit code vastgesteld	Vijf digitale lijnsensoren; zwart wordt 1 in een vijftekenspatroon. PWM-basis 0.35 is aansturingsniveau, geen gemeten snelheid. [B02/B04]
Nog niet aangeleverd	Inhoud van robot_mqtt.py en objectdetectie.py; exact ESP32-model, MicroPython-build en MQTT-clientversie.
Nog te meten	Afstanden, lijnbreedte, sensorafstand tot wielas, sensorhoogte/polariteit, motorrespons en voedingsprofielen.
Nog te besluiten	Servicepunt per attractie: terminal Sxx of nabijgelegen Jxx; herkenbare ankers/markers en toegestane manoeuvres.
Ontwerpvoorstel	Python-kern met tick-state-machine; FastAPI + browser-SVG; MQTT 3.1.1-baseline en versiegebonden JSON-contracten.

Binnen de eerste versie

Een read-only kaart, routeplanning voor geordende unieke attracties, terugkeer naar depot, configureerbare verblijftijd, simulatieruns, sensorreplay, foutinjectie, live status en reproduceerbare logs. Zowel normale opdrachten als wijzigingen tijdens rijden maken deel uit van het contract.

Buiten de eerste versie

Vrije verkenning van een onbekende baan, autonoom SLAM, meerdere shuttles met verkeersregeling, automatische batterijcompensatie zonder metingen, publiek internetbeheer en automatische fysieke uitrol. Herhaald bezoek aan dezelfde attractie in één missie is voor v1 ongeldig; uitbreiding vereist een expliciete nieuwe contractkeuze.

Definitie van alle routes

Het systeem berekent een route op aanvraag. Het schrijft niet alle mogelijke wandelingen met willekeurige rondjes vooraf uit. Voor deze kaart zijn er 325 geordende selecties van één t/m vijf verschillende attracties; die vormen een eindige, volledige basisset voor de gekozen missiefunctionaliteit.

02.1 Functionele en technische eisen

REQ-identifiers worden gekoppeld aan acceptatiecriteria en testscenario's.

ID	Eis	Uitwerking
REQ01	Eén volledige versie van de kaart	Alle lijnen en vijf attracties uit de foto, depot inbegrepen; gedeelde bron voor planner, simulator en robot.
REQ02	Variabele bestemmingen	Een geordende selectie van attracties wordt op aanvraag gepland; geen nieuwe main.py per rit.
REQ03	Rijrichting en terugweg	Inkomende verbinding, lokale poorten, bochten en vertrekmanoeuvres bepalen de route.
REQ04	Eén uitvoeringsmodel	Dezelfde RobotAgent/RouteExecutor voor simulatie en ESP32, met verwisselbare adapters.
REQ05	Betrouwbare waarneming	Vijf bits, debounce, lijnverlies, expliciete onzekerheid en geen dubbele kruispunttelling.
REQ06	MQTT-opdrachtcyclus	Ontvangen, geaccepteerd, toegepast en uitgevoerd zijn verschillende bevestigingen.
REQ07	Wijziging tijdens rijden	Nieuwe revisie ontvangen tijdens rijden; atomair toepassen op vooraf afgesproken bevestigde grens.
REQ08	Geen fictieve positie	Een verwacht punt is geen gemeten uniek punt; onzekerheid en herlokalisatie blijven zichtbaar.
REQ09	Bediening en observatie	Kaart, route, rijrichting, status, revisie en actualiteit zichtbaar zonder brokerwachtwoorden in de browser.
REQ10	Reproduceerbaarheid	Kaart/config/plan/seed en ruwe opnamen maken verschillen terug te vinden en herspeelbaar.

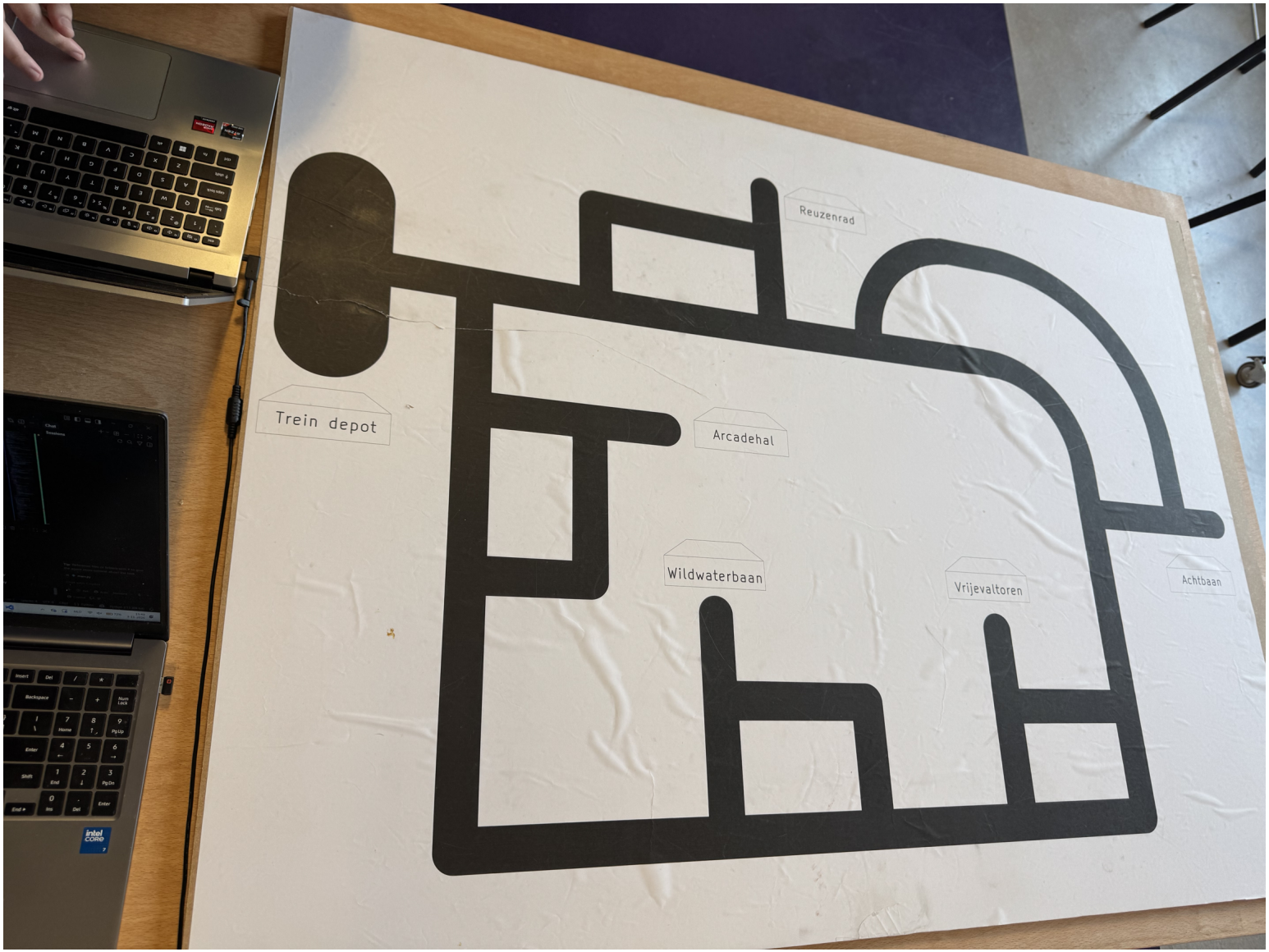
02.2 Functionele en technische eisen

REQ-identifiers worden gekoppeld aan acceptatiecriteria en testscenario's.

ID	Eis	Uitwerking
REQ11	Scheiding sim en hardware	Een simulatiebericht kan geen fysieke motoropdracht worden.
REQ12	MicroPython-overdraagbaarheid	De gedeelde kern heeft geen desktopframeworks, dataclasses of runtime typing-afhankelijkheid.
REQ13	Reactietijd	Geen sleep voor een hele bocht/halte; netwerk en logging mogen de regellus niet blokkeren.
REQ14	Versies en configuratie	Kaart, contract, software, board, pinnen en kalibratie zijn identificeerbaar en controleerbaar.
REQ15	Aantoonbaar succes	Alle gevraagde bezoeken, servicebeleid en eindpunt bevestigd voor een positief resultaat.
REQ16	Beveiligde toegang	Broker-ACL, autorisatie, geheimen buiten broncode en gecontroleerde hardware-activering.
REQ17	Meetbare kwaliteit	Traceerbare acceptatiecriteria en testorakels voor software, integratie en fysieke baan.
REQ18	Verbindings-/herstartgedrag	Geen dubbele rit na reconnect/reset; vooraf vastgesteld offline- en herstelbeleid.
REQ19	Dynamische verblijftijden	0 ms bij een doorrit of 5000 ms bij een halte; geen verborgen meetstop.
REQ20	Bewijs en vrijgave	Ontwerpsimulatie, opgenomen sensortests en fysieke acceptatie hebben aparte vrijgavepoorten.

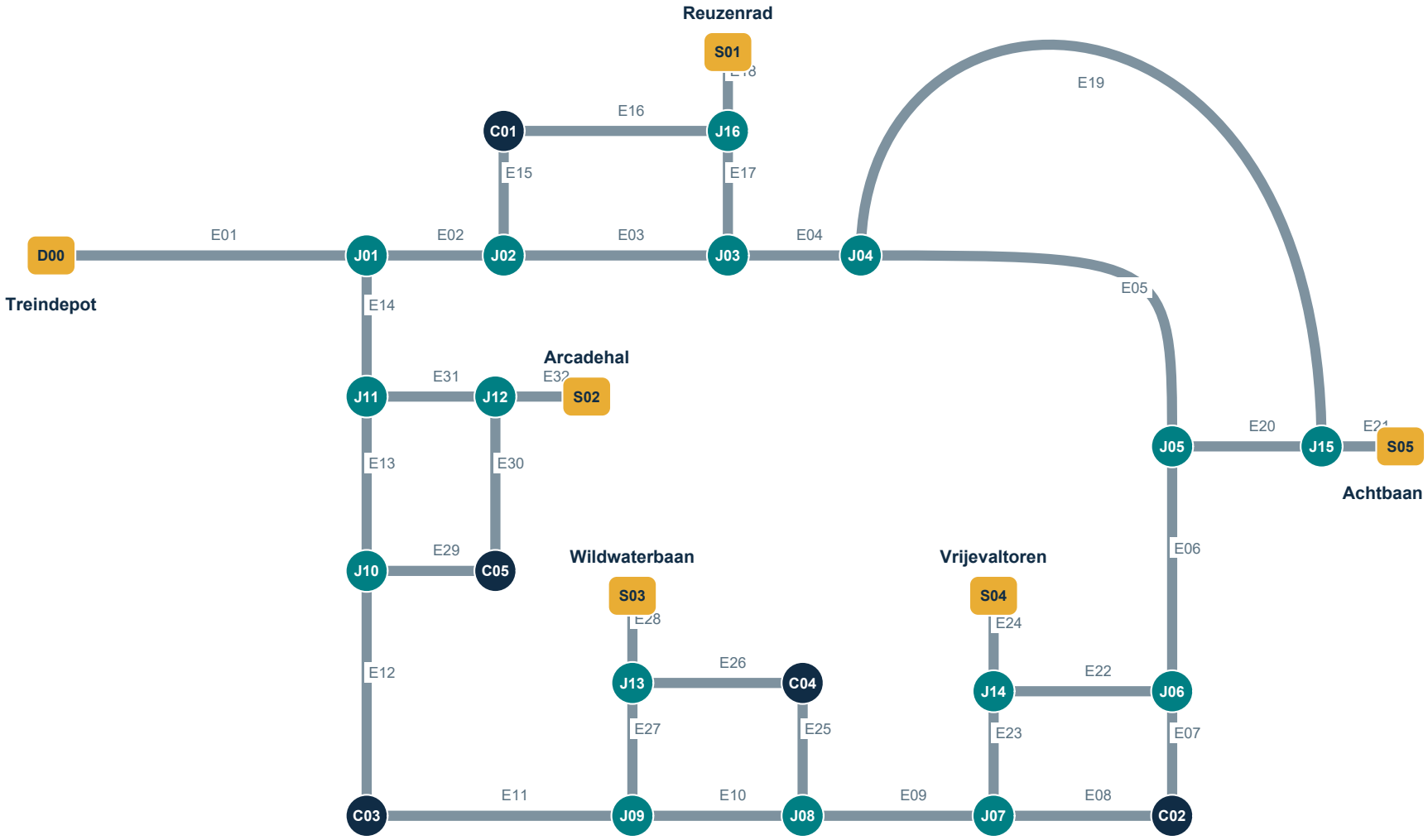
03 Bronfoto van het parcours

Aangeleverd bovenaanzicht, alleen voor leesrichting gedraaid. Geen lengtemeting of servicevalidatie.



04 Digitale topologie en vaste IDs

Schematische hertekening op basis van de foto. Lokale poorten zijn N/E/S/W in deze tekening; geen kompasmeting.



D = depot; J = keuzepunt; C = verplichte hoek; S = getekend eindpunt bij attractie. Een ronde doorgaande bocht is edge-geometrie, geen extra kruispunt.

27 punten | 32 verbindingen | 6 eindpunten | 6 lussen | 112 lokale overgangen. Service-/ankerplaatsen en werkelijke afstanden zijn nog te valideren.

04.1 Catalogus van alle punten

Identieke IDs in kaartbestand, routeplan, log, tests en UML-objecten.

ID	Soort	Betekenis	Poorten
D00	DEPOT	Treindepot	E
J01	JUNCTION	Depot-aansluiting	W/E/S
J02	JUNCTION	Reuzenrad-lus west	W/E/N
J03	JUNCTION	Reuzenrad-stam	W/E/N
J04	JUNCTION	Buitenbocht-keuze	W/E/N
J05	JUNCTION	Achtbaan / rechterrand	N/E/S
J06	JUNCTION	Vrijeval-zijpad	N/S/W
J07	JUNCTION	Vrijeval / onderrand	W/E/N
J08	JUNCTION	Wildwater-zijpad / onderrand	W/E/N
J09	JUNCTION	Wildwater-stam / onderrand	W/E/N
J10	JUNCTION	Arcade-teruglus	N/S/E
J11	JUNCTION	Arcade-inrit	N/S/E
J12	JUNCTION	Arcade-T	W/E/S
J13	JUNCTION	Wildwater-T	N/S/E
J14	JUNCTION	Vrijeval-T	N/S/E
J15	JUNCTION	Achtbaan-T	N/E/W
J16	JUNCTION	Reuzenrad-T	N/S/W
C01	CORNER	Hoek Reuzenrad-lus	S/E
C02	CORNER	Hoek rechtsonder	N/W
C03	CORNER	Hoek linksonder	N/E
C04	CORNER	Hoek Wildwater-zijpad	S/W
C05	CORNER	Hoek Arcade-lus	N/W
S01	TERMINAL	Reuzenrad	S
S02	TERMINAL	Arcadehal	W
S03	TERMINAL	Wildwaterbaan	S
S04	TERMINAL	Vrijevaltoren	S
S05	TERMINAL	Achtbaan	W

N/E/S/W beschrijven lokale aansluitingen in de getekende oriëntatie, niet de magnetische rijrichting. Rondlopende edges kunnen aan twee kanten dezelfde lokale poortnaam hebben.

In JSON zijn nodes, edges en stations lijsten van records. De loader bouwt de dict-indexen uit het klassendiagram op hun unieke IDs. Edge-geometrie bevat een kind en punten in schematische eenheden; metadata markeert de kaart als niet fysiek gevalideerd.

04.2 Catalogus van alle verbindingen

Elke edge is voorlopig bidirectioneel; length_mm=null tot inmeting. E05 en E19 zijn verschillende gebogen routes.

Edge	Van / poort	Naar / poort	Geometrie
E01	D00 / E	J01 / W	recht / hoek via aparte C-node
E02	J01 / E	J02 / W	recht / hoek via aparte C-node
E03	J02 / E	J03 / W	recht / hoek via aparte C-node
E04	J03 / E	J04 / W	recht / hoek via aparte C-node
E05	J04 / E	J05 / N	binnenbocht: J04 E naar J05 N
E06	J05 / S	J06 / N	recht / hoek via aparte C-node
E07	J06 / S	C02 / N	recht / hoek via aparte C-node
E08	C02 / W	J07 / E	recht / hoek via aparte C-node
E09	J07 / W	J08 / E	recht / hoek via aparte C-node
E10	J08 / W	J09 / E	recht / hoek via aparte C-node
E11	J09 / W	C03 / E	recht / hoek via aparte C-node
E12	C03 / N	J10 / S	recht / hoek via aparte C-node
E13	J10 / N	J11 / S	recht / hoek via aparte C-node
E14	J11 / N	J01 / S	recht / hoek via aparte C-node
E15	J02 / N	C01 / S	recht / hoek via aparte C-node
E16	C01 / E	J16 / W	recht / hoek via aparte C-node
E17	J16 / S	J03 / N	recht / hoek via aparte C-node
E18	J16 / N	S01 / S	recht / hoek via aparte C-node
E19	J04 / N	J15 / N	bovenste buitenbocht: J04 N naar J15 N
E20	J15 / W	J05 / E	recht / hoek via aparte C-node
E21	J15 / E	S05 / W	recht / hoek via aparte C-node
E22	J06 / W	J14 / E	recht / hoek via aparte C-node
E23	J14 / S	J07 / N	recht / hoek via aparte C-node
E24	J14 / N	S04 / S	recht / hoek via aparte C-node
E25	J08 / N	C04 / S	recht / hoek via aparte C-node
E26	C04 / W	J13 / E	recht / hoek via aparte C-node
E27	J13 / S	J09 / N	recht / hoek via aparte C-node
E28	J13 / N	S03 / S	recht / hoek via aparte C-node
E29	J10 / E	C05 / W	recht / hoek via aparte C-node
E30	C05 / N	J12 / S	recht / hoek via aparte C-node
E31	J12 / W	J11 / E	recht / hoek via aparte C-node
E32	J12 / E	S02 / W	recht / hoek via aparte C-node

04.3 Servicepunten en meetplan

Het verschil tussen een lijneinde, een kruispunt en een bevestigde halte.

Station-ID	Attractie	Getekend einde	Nabijgelegen T
FERRIS	Reuzenrad	S01	J16
ARCADE	Arcadehal	S02	J12
WILDWATER	Wildwaterbaan	S03	J13
FREEFALL	Vrijevaltoren	S04	J14
COASTER	Achtbaan	S05	J15

De tabel legt terminal_node_id en een kandidaat-aansluiting vast. service_node_id blijft null en hardware_validated=false totdat de plaats op de baan is bevestigd. Het ontwerp kiest dus niet ongemerkt een T in plaats van het lijneinde. Voor ontwerpsimulaties wordt het getekende eindpunt als nominale bestemming gebruikt; de interface toont die keuze als nog niet fysiek gevalideerd.

Inmeting en besluit per station

- Geef een fysiek punt aan waar aankomst wordt gemeten en waar de robot tijdens een halte staat.
- Leg het toegestane aankomstpad en alle geldige vertrekpaden vast, inclusief wel/geen U-bocht.
- Bepaal welk anker die plaats uniek maakt: toegestane marker/code of een expliciet bevestigde operatorhandeling.
- Meet de sensor-/wielasafstand en draaigeometrie; koppel kalibratie aan het voertuig en de voeding.
- Leg voor iedere stationconfiguratie bewijs, datum, beoordelaar en map_version vast.

Waarom ankers nodig zijn

Vijf lijnsensorbits identificeren doorgaans geen uniek kruispunt. Een passend patroon is hoogstens consistent met de verwachte locatie. Zonder unieke positie-evidence blijft de plaats ESTIMATED; een onzekere rit mag niet als autonoom succesvol worden afgesloten. Een begeleide proef heeft een expliciet supervised resultaat.

05 Routeplanning met aankomstrichting

Algoritme, overgangen en grenzen

De planner zoekt op toestand (node_id, incoming_edge_id). Een actie kiest een uitgaande edge en de bijbehorende TurnRule. Met alleen een node-ID weet de planner niet of dezelfde tak links of rechts ligt. Voor elke edge worden daarom de lokale poorten aan beide uiteinden opgeslagen, ook bij de twee bochten bovenaan.

Aankomstpoort	Uitgang	Actie
W	E / N / S	STRAIGHT / LEFT / RIGHT
E	W / S / N	STRAIGHT / LEFT / RIGHT
N	S / E / W	STRAIGHT / LEFT / RIGHT
S	N / W / E	STRAIGHT / LEFT / RIGHT
dezelfde poort	alleen terminal/depot	U_TURN, na expliciete service-/vertrekbeslissing

Kosten en determinisme

Zolang lengtes onbekend zijn, gebruikt de planner hopkosten: elke edge kost 1. Dat is de route met het kleinste aantal verbindingen, niet noodzakelijk de kortste afstand. Afstandsmodus wordt pas toegestaan als alle gebruikte length_mm-waarden gemeten en niet-negatief zijn. Dijkstra gebruikt een priority queue; gelijke kosten worden deterministisch op edge-ID-pad beslist. [B09]

Missie samenstellen

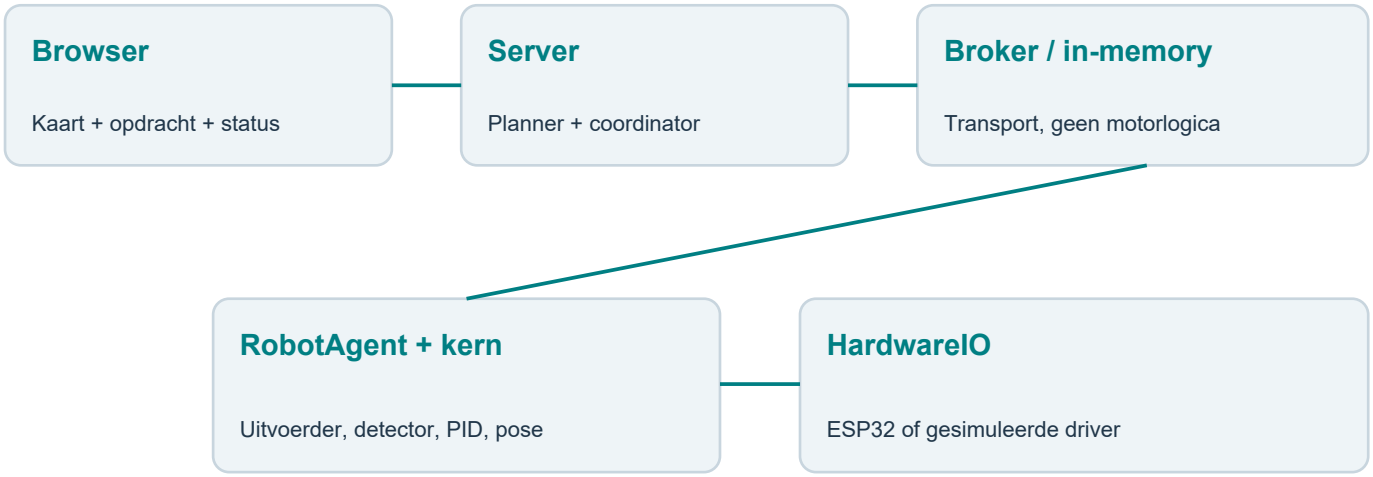
Plan per gewenst station een pad en behoud de inkomende edge voor het volgende traject. Voeg bij elk station een expliciete SERVICE-stap met dwell en aankomstevidence toe. De volgende TRANSIT-stap kan een U_TURN zijn als het gekozen servicepunt een echt eindpunt is. Eindig met een END-stap op het gevraagde eindanker; het depot is de standaard eindbestemming.

Ontwerpcontrole uitgevoerd

702 nodeparen, 1728 gerichte aankomst/doelcombinaties en 325 attractievolgorden zijn structureel gecontroleerd. Maximaal 49 transit-edges in deze basisset; met 5 services en END maximaal 55 stappen. Dit is een controle van het model, niet van toekomstige productcode of fysieke sensoren.

06 Architectuur en verantwoordelijkheden

Eén kaart, één uitvoerder en expliciete grenzen



De server bezit de gewenste missie en RoutePlanner. MissionCoordinator verstuurt een versiegebonden plan en spiegelt alleen valide gebeurtenissen van de robot. Hij bestuurt geen motoren en verklaart een rit niet zelf succesvol omdat het animatiepijltje het eindpunt bereikt.

RobotAgent is de gezaghebbende robotcyclus. Hij ontvangt opdrachten, beheert revisies en servicehistorie, laat sensoren filteren en roept RouteExecutor per tick aan. RouteExecutor gebruikt PoseEstimator, LineController, HardwareIO en Clock. Simulator en ESP32 draaien dezezelfde kern; alleen de adapters verschillen.

In simulatie verbinden twee InMemoryChannel-einden server en RobotAgent. Met een fysieke robot lopen dezelfde berichten via MqttGateway, broker en FirmwareMqttChannel. Netwerkcallbacks zetten uitsluitend data in een begrensde queue. De kern wordt door één tick-eigenaar gewijzigd; callbacks mogen niet tegelijk motoren of route-index aanpassen. [B06]

Afhankelijkheden die als UML-associatie zijn getekend, worden via constructors geinjecteerd; ze zijn niet allemaal nogmaals als los attribuut herhaald. EventJournal.append schrijft eerst naar RAM. De server bewaart via een worker; firmware checkpoint alleen buiten beweging. De server maakt kritieke events duurzaam voordat hij event_ack stuurt.

Platformkeuze

Voorgesteld: CPython-server met FastAPI/HTTP/WebSocket en Paho-MQTT; browser met gewone HTML/CSS/JavaScript en SVG. De kern gebruikt eenvoudige Python-klassen en JSON-compatibele waarden. Typeaanduidingen in UML zijn documentatie; de firmware hoeft geen typing, enum, dataclasses of desktopframeworks te importeren. [B06/B07]

07.1 Voorgestelde mappen en bestanden

Nieuwe projectstructuur; dit document heeft deze applicatie nog niet geimplementeerd.

```
shuttle-digital/
|-- README.md                # installatie, runnen, vrijgave
|-- pyproject.toml           # CPython tooling en projectmetadata
|-- requirements.lock        # bij implementatie exact vastzetten
|-- .env.example             # namen van instellingen, geen geheimen
|-- .gitignore               # .env, opnamen, secrets, lokale runs
|-- shared/
|   |-- __init__.py
|   |-- ports.py             # Clock, HardwareIO, MessageChannel, EventSink
|   |-- journal.py           # EventJournal, RAM + uitgestelde opslag
|   |-- protocol.py          # serialisatie, versie, foutcodes
|   |-- model/
|   |   |-- enums.py         # stringconstanten, geen enum-import
|   |   |-- map.py           # TrackMap, TrackNode, TrackEdge, TurnRule
|   |   |-- station.py       # Station
|   |   |-- mission.py       # Mission
|   |   |-- plan.py          # RoutePlan, RouteStep
|   |   |-- pose.py          # RobotPose
|   |   |-- observations.py  # SensorFrame, TrackObservation
|   |   `-- messages.py      # RobotCommand, RobotEvent
|   `-- core/
|       |-- validation.py    # GraphValidator
|       |-- planner.py       # RoutePlanner; alleen server hoeft te plannen
|       |-- commands.py      # CommandValidator
|       |-- coordinator.py   # MissionCoordinator op server
|       |-- agent.py         # RobotAgent op simulator/ESP32
|       |-- localization.py  # PoseEstimator
|       |-- detector.py      # JunctionDetector
|       |-- line_control.py  # LineController
|       `-- executor.py      # RouteExecutor
|-- contracts/schemas/
|   |-- map.schema.json
|   |-- mission.schema.json
|   |-- command.schema.json
|   |-- plan.schema.json
|   `-- event.schema.json
|-- maps/
|   |-- park_v1.json         # enige bewerkbare topologische bron
|   |-- park_v1.svg          # gegenereerd uit JSON, geen tweede kaart
|   `-- survey.json          # metingen/evidence met map_version
|-- config/
|   |-- sim.json             # seed, tick, ruis, testprofielen
|   |-- hardware.example.json # board/pinnen/kalibratie zonder credentials
|   `-- broker.example.json  # topicprefix en connectie-instellingen
```

Map- en contractgeneratie gebeurt op de computer. De ESP32 krijgt alleen de benodigde kern, adapters, goedgekeurde kaart en configuratie; geen server-, test- of webafhankelijkheden.

07.2 Voorgestelde mappen en bestanden

Nieuwe projectstructuur; dit document heeft deze applicatie nog niet geimplementeerd.

```
shuttle-digital/
|-- server/
|   |-- main.py               # compositie, dependencies en shutdown
|   |-- api.py                # ApiService
|   `-- adapters/
|       |-- clock.py          # SystemClock
|       `-- mqtt_gateway.py   # MqttGateway, I/O-worker
|-- firmware/
|   |-- boot.py               # motoren uit, boardconfig laden
|   |-- main.py               # composeer agent en korte ticks
|   `-- adapters/
|       |-- clock.py          # Esp32Clock
|       |-- hardware.py       # Esp32Hardware
|       `-- mqtt_channel.py   # FirmwareMqttChannel
|-- sim/
|   |-- engine.py             # SimulationEngine
|   |-- hardware.py           # SimHardware
|   |-- clock.py              # FakeClock
|   |-- channel.py            # InMemoryChannel
|   |-- faults.py             # FaultInjector
|   `-- replay.py             # ReplayRunner
|-- web/
|   |-- index.html
|   |-- styles.css
|   |-- client.js             # BrowserClient
|   |-- map_renderer.js       # MapRenderer
|   `-- telemetry_store.js    # TelemetryStore
|-- tests/
|   |-- unit/                 # graph, planner, detector, PID, contract
|   |-- integration/          # MQTT, API, revision, UI, namespaces
|   |-- scenarios/            # T01-T48 machineleesbare fixtures
|   |-- recordings/           # kleine geanonimiseerde golden traces
|   `-- hardware/             # meetprotocol en acceptatierapporten
|-- scripts/
|   |-- validate_map.py       # topologie en schema controleren
|   |-- generate_assets.py    # SVG/firmwaredata uit dezelfde bron
|   |-- check_contracts.py    # CPython/MicroPython golden vectors
|   `-- run_checks.py         # herhaalbare test-entrypoint
|-- docs/
|   |-- architecture.md       # beslissingen en interfaces
|   |-- commissioning.md      # survey, pinnen, markers, voeding
|   `-- operations.md         # starten, incidenten, recovery, rollback
`-- storage/runs/             # gitignored JSONL, ruwe frames, snapshots
```

Map- en contractgeneratie gebeurt op de computer. De ESP32 krijgt alleen de benodigde kern, adapters, goedgekeurde kaart en configuratie; geen server-, test- of webafhankelijkheden.

08.1 Volledig klassenregister

Alle 42 klassen zijn gekoppeld aan een bestand en verantwoordelijkheid. UML-attributen, methoden en relaties staan in bijlage A/B.

ID / klasse	Bestand	Verantwoordelijkheid
C01 TrackMap	shared/model/map.py	Canonical kaart; topologie en expliciete lokale poorten, geen afslaglijst per proefrit.
C02 TrackNode	shared/model/map.py	Kruispunt, verplichte hoek, eindpunt of depot; coordinaten zijn schematisch.
C03 TrackEdge	shared/model/map.py	Verbinding met lokale eindtangenten. Lengte onbekend blijft null; geen verzonnen mm.
C04 TurnRule	shared/model/map.py	Afslag hangt af van aankomstrichting. Ook verplichte hoeken en terminal-U-bochten zijn expliciet.
C05 Station	shared/model/station.py	Scheidt getekend lijneinde van het werkelijk goedgekeurde aankomst-/vertrekpunt.
C06 Mission	shared/model/mission.py	Geordende bestemmingen en verblijftijden; geen impliciet sorteren of samenvoegen.
C07 RoutePlan	shared/model/plan.py	Versiegebonden uitvoerplan; inclusief verwachte aankomst, service en eindactie.
C08 RouteStep	shared/model/plan.py	Edge-stap met STRAIGHT/LEFT/RIGHT/U_TURN, of SERVICE/END; hoeken worden niet stilzwijgend weggelaten.
C09 RobotPose	shared/model/pose.py	Operationele plaatsinschatting; verwachte positie wordt nooit als gemeten waarheid gepresenteerd.
C10 SensorFrame	shared/model/observations.py	Ruwe genormaliseerde meting; precies vijf lijnsensorbits, zwart=1.
C11 TrackObservation	shared/model/observations.py	Gefilterde waarneming; compatibiliteit is geen unieke plaatsbepaling zonder anker.
C12 RobotCommand	shared/model/messages.py	Gevalideerde opdracht met ID, revisie, geldigheid en bekende toepassingsgrens.
C13 RobotEvent	shared/model/messages.py	Status/voortgang/resultaat met bron simulation of hardware; geen impliciete MQTT-ACK als ritsucces.
C14 GraphValidator	shared/core/validation.py	Controleert IDs, verbindingen, poorten, verboden overgangen en oningevulde fysieke servicepunten.

08.2 Volledig klassenregister

Alle 42 klassen zijn gekoppeld aan een bestand en verantwoordelijkheid. UML-attributen, methoden en relaties staan in bijlage A/B.

ID / klasse	Bestand	Verantwoordelijkheid
C15 RoutePlanner	shared/core/planner.py	Dijkstra op toestand (knoop, inkomende edge), met deterministische gelijke-kostenkeuze.
C16 CommandValidator	shared/core/commands.py	Controleert vorm, duplicaten, revisies en voorwaarden; firmware gebruikt compacte contractvalidatie.
C17 MissionCoordinator	shared/core/coordinator.py	Server: gewenste missie plannen en gevalideerde robotresultaten spiegelen; bestuurt geen motoren.
C18 PoseEstimator	shared/core/localization.py	Beheert onzekerheid; gemiste/dubbele punten mogen geen fictieve bevestigde locatie worden.
C19 JunctionDetector	shared/core/detector.py	Debounce en precies-een-punt-per-passage; een bevestigde lijn niet nogmaals blokkeren.
C20 LineController	shared/core/line_control.py	Tijdgebaseerde PID met begrensde uitgangen; geen routekeuzes of netwerk-I/O.
C21 RouteExecutor	shared/core/executor.py	Niet-blokkerende uitvoering: lijnvolgen, bocht, service, zoeken en einde; motorbesturing via port.
C22 Clock	shared/ports.py	Interface voor monotone tijd, inclusief ticks-wrap op ESP32.
C23 HardwareIO	shared/ports.py	Interface voor sensoren en motoren; uitvoerder kent geen GPIO-pinnen.
C24 MessageChannel	shared/ports.py	Bidirectionele interface met begrensde verwerking; richting/type wordt aan de grens gevalideerd.
C25 EventSink	shared/ports.py	Interface voor journal en replay; kritieke gebeurtenissen niet stilzwijgend verliezen.
C26 SystemClock	server/adapters/clock.py	CPython monotone tijd.
C27 FakeClock	sim/clock.py	Deterministische simulatietijd, onafhankelijk van browser-FPS.
C28 Esp32Clock	firmware/adapters/clock.py	MicroPython ticks_ms/ticks_diff-adapter.

08.3 Volledig klassenregister

Alle 42 klassen zijn gekoppeld aan een bestand en verantwoordelijkheid. UML-attributen, methoden en relaties staan in bijlage A/B.

ID / klasse	Bestand	Verantwoordelijkheid
C29 Esp32Hardware	firmware/adapters/hardware.py	Normaliseert vijf sensoren, PWM, motorpolariteit en optionele afstand-/ankermeting.
C30 MqttGateway	server/adapters/mqtt_gateway.py	Paho-client in I/O-worker; brokercallbacks zetten alleen berichten in een queue.
C31 FirmwareMqttChannel	firmware/adapters/mqtt_channel.py	ESP32 MQTT-adapter; concrete client/polling moet eerst op blokkering en budget getest worden.
C32 SimHardware	sim/hardware.py	Implementeert HardwareIO. Ground truth blijft verborgen voor de uitvoerder.
C33 FaultInjector	sim/faults.py	Herhaalbare ruis, gemiste punten, dubbele berichten en disconnects.
C34 SimulationEngine	sim/engine.py	Een eigenaar van simulatietoestand; bounded test-run is iets anders dan robotzoektijd.
C35 EventJournal	shared/journal.py	Begrensd RAM-journal; server schrijft via worker, firmware checkpoint alleen buiten beweging. JSONL met versies/seed.
C36 ReplayRunner	sim/replay.py	Herspeelt ruwe frames en opdrachten op FakeClock; een eventlog alleen herspeelt geen sensoren.
C37 ApiService	server/api.py	HTTP/WebSocket-grens, validatie en expliciete keuze simulation versus hardware.
C38 BrowserClient	web/client.js	Bediening; nooit directe brokercredentials of motorcommando vanuit de browser.
C39 MapRenderer	web/map_renderer.js	SVG-kaart, richtingpijl, verwachte/zekere positie en punten met onbekende kalibratie.
C40 TelemetryStore	web/telemetry_store.js	Voorkomt dat een oude browserweergave of ontbrekend event als actuele toestand wordt getoond.
C41 RobotAgent	shared/core/agent.py	Gezaghebbende robotcyclus op ESP32 of in simulatie; netwerk naar uitvoerder, bezoekregistratie en resultaat.
C42 InMemoryChannel	sim/channel.py	Twee verbonden kanaaleinden voor planner en gesimuleerde robot, zonder echte broker.

09 Toestanden en uitvoeringsinvarianten

De robot reageert per tick; er is geen volledige bocht of halte in een blokkerende while-lus.

Toestand	Gedrag	Verlaten wanneer
IDLE	Motoren uit; opdrachten en ankers ontvangen.	START geaccepteerd en beginanker geldig.
FOLLOW	Ruwe frames lezen, PID en verwachte edge volgen.	Breed punt, lijnverlies, obstakel of servicepunt.
APPROACH	Patroon bevestigen tijdens bewegen; verwachte poorten toetsen.	Passage/manoeuvre voldoende onderbouwd.
TURN	Gewenste afslag uitvoeren met korte updates.	Nieuwe lijn bevestigd, passend bij geplande overgang.
SERVICE	Aankomstbewijs controleren; dwell met deadline.	Dwell verstreken en volgende vertrekactie toepasbaar.
SEARCH	Lijn/anker zoeken volgens beleid; missie UNCERTAIN.	Vereiste herlokalisatie-evidence beschikbaar.
OBSTACLE	Motoren uit; afstand/clearance blijven meten.	Obstakel weg en hervatten toegestaan.
PAUSED	Motoren uit; plan, stap en resume_state behouden.	Geautoriseerde RESUME en geen obstakel/fault.
COMPLETE	END uitgevoerd, eindanker en bezoekhistorie correct.	Nieuw expliciet geaccepteerd mission_id.
FAULT	Motoren uit, diagnose en herstelpad beschikbaar.	Gecontroleerd herstel; nooit stil als COMPLETED labelen.

Invarianten

- Een stap wordt niet verbruikt door tijdsverloop of enkel omdat een draai gestart is.
- Bevestigde uitgangsevidence wordt eenmaal gebruikt voor vrijgave; geen dubbele wachttijd.
- Ieder node-/service-event heeft een uniek event_id en een monotone sequence.
- SEARCH zonder timeout mag, maar geeft geen betrouwbare kaartpositie of automatisch ritsucces.
- Alle gevraagde services en het eindanker moeten bewezen zijn voor COMPLETED.

10 Timing, sensoren en motoren

De kaartplanner lost geen accuspanning of geblokkeerde sensormeting op.

RobotAgent.tick leest tijd en een SensorFrame, verwerkt een begrensd aantal opdrachten, laat JunctionDetector en PoseEstimator bijwerken, roept RouteExecutor aan en zet begrensde motorcommando's. Netwerk-I/O en bestandsflush horen niet in deze kritieke stap. Een wachttijd van 5000 ms wordt een SERVICE-deadline die iedere tick wordt gecontroleerd; MQTT blijft ondertussen bereikbaar.

Onderdeel	Ontwerpafspraak
Regellus	Nominaal tickdoel 5 ms. Werkelijke sample->motorlatentie wordt gemeten; CPU-wachttijd is geen bewijs van 200 Hz.
Detector	Gebruik monotone tijdvensters en sample_seq, geen vast aantal frames als vervanging voor echte tijd.
PID	Bereken met werkelijk dt; beperk integraal en motoruitgangen. Reset bij manoeuvre-/herstelgrens. Bestaande gains niet blind naar andere dt-formule kopieren.
Voeding	PWM 0.35 is een duty-aansturing. Voor constante fysieke snelheid zijn meting/kalibratie en eventueel terugkoppeling nodig. [B04]
Tijd	Esp32Clock gebruikt ticks_diff; trek ticks niet rechtstreeks af. UTC voor expiry staat los van monotone besturingstijd. [B05]
Queues	Begindoelen: inbox 32, kritieke outbox 64, telemetrie coalescing. Definitief pas na RAM-/belastingmeting.
Opname	Ruwe frames in labmodus opnemen of korte ringbuffer; geen onbeperkt flashschrijven in de rijlus.
Zoeken	search_timeout_ms=null kan onbeperkt zoeken toestaan. Bediening toont UNCERTAIN; routewissel wacht op anker.

Nog te meten

Lineaire afstanden, lijnbreedte, sensorhoogte, sensor-/wielasafstand, motordodezone, voeding versus snelheid en draaihoek versus tijd. De oude 40 ms/700 ms/500 ms mogen niet als universele voertuigconstanten in het kaartmodel worden vastgelegd.

Reproduceerbaar S1-startprofiel: tick 5 ms, breedbevestiging 10 ms, uitgang 25 ms, stale-framegrens 50 ms; maximaal 2 kanaalberichten per tick. Referentieseeds 11/22/33. T11 gebruikt per 5 ms de reeks 00100 x8, 00111 x3, 00100 x6. Dit zijn voorgestelde synthetische beginwaarden, geen nieuwe hardwarekalibratie; ze staan ook in de PDF-bijlage sim_profile.json.

11 MQTT-topics en bevestigingen

Voorgestelde baseline: MQTT 3.1.1; concrete ESP32-clientcapaciteiten eerst controleren. [B03/B06]

Topic onder shuttle/v1/{env}/{robot_id}/	Richting	QoS / retain
command	server -> robot	1 / false
ack	robot -> server	1 / false
event	robot -> server	1 / false; kritieke events met application-ACK
event_ack	server -> robot	1 / false
state	robot -> server	1 / true; versiegebonden snapshot
telemetry	robot -> server	0 / false; verlies/coalescing toegestaan
availability	robot/broker -> server	1 / true; LWT offline

env is sim of hw. Daarnaast bevat elke envelope source en robot_id; de ontvanger controleert die waarden opnieuw. De browser gebruikt alleen de API en WebSocket. Hardware-ingangen hebben eigen credentials en topic-ACL. START en REPLACE zijn nooit retained. Een retained state is slechts een snapshot, geen nieuwe uitvoeropdracht.

MessageChannel levert body plus transportmetadata (topic en retained-flag) aan de grens. CommandValidator controleert die metadata buiten het JSON-payloadschema. De concrete MicroPython-client moet deze informatie beschikbaar maken; dat is nog te onderzoeken, naast de sessie-/expirybescherming.

Vier verschillende bevestigingen

Disposition	Wat het betekent
RECEIVED	Envelope ontvangen; nog geen belofte van geldige route of uitvoering.
ACCEPTED	Contract, kaart, sessie, revisie, capaciteit en toepassingsgrens akkoord.
DEFERRED / APPLIED	Wacht op de afgesproken grens / nieuwe revisie is daadwerkelijk actief.
COMPLETED event	Gehele missie inclusief services en eindanker bevestigd; anders FAILED/CANCELLED/UNCERTAIN.

MQTT-QoS bevestigt berichttransport. De eigen opdracht-/resultaatcyclus bevestigt applicatiegedrag. QoS1 kan duplicaten leveren; ID/revisie en event-deduplicatie zijn daarom onderdeel van het ontwerp. [B03]

De browser hergebruikt een Idempotency-Key bij een HTTP-retry. MissionCoordinator bewaart de key, bodyhash en mission_id in het serverjournal en herstelt die index bij serverstart. Dezelfde key met een andere body geeft een conflict. Nog geldige dedup-records worden niet wegens cachecapaciteit verwijderd; bij volle capaciteit wordt een nieuwe opdracht expliciet afgewezen.

11.1 Opdracht- en plancontract

UTF-8 JSON, schema_version=1, additionalProperties=false aan de buitenste grenzen. [B08]

Veld	Type / grens	Semantiek
schema_version	int = 1	Onbekende versie afwijzen.
command_id	string, 1..64	Uniek; retries houden hetzelfde ID.
session_id	string, 1..64	Actuele boot-/handshakesessie, voorkomt replay na reset.
kind	CommandKind	START/REPLACE/PAUSE/RESUME/CANCEL/CONFIRM_ANCHOR.
mission_id / revision	string <=64 / int >=1	REPLACE verhoogt revisie van dezelfde missie.
map_version	string <=64	Exact gelijk aan goedgekeurde kaart op robot.
issued_at / expires_at	UTC RFC3339 strings	Geldigheid voor accepteren, niet automatisch een rittime-out.
valid_from_node	node-ID of null	REPLACE: afgesproken volgende ankergrens; START: bekend beginanker.
source / robot_id	enum / string <=32	Bron/modus en bedoelde ontvanger controleren.
payload	object per kind	START/REPLACE: Mission + RoutePlan. Overige opdrachten hebben alleen hun benoemde parameters.

Mission en RoutePlan

Mission bevat mission_id, revision, visit_ids (1..5 unieke station-IDs), dwell_ms (gelijke lengte, elk 0..300000), end_node_id en status. De planner bewaart de volgorde. RoutePlan bevat plan_id, plan_hash, mission_id, revision, map_version, start_pose en steps. Een RouteStep bevat step_id, index, from_id, to_id, edge_id (of null), maneuver, station_id (of null), dwell_ms en evidence.

De manoeuvre hoort bij vertrek uit from_id via edge_id, gezien de inkomende edge uit de pose. SERVICE gebruikt from_id=to_id en geen edge; END heeft geen volgende edge. Hoeken staan als expliciete vertrekactie in de edge-stap. Een terminal-U-bocht hoort bij de eerste edge na SERVICE.

Capaciteit en klok

Ontwerpgrenzen: maximaal 64 stappen en 16384 UTF-8 bytes per planbericht. De kaartbasis heeft maximaal 55 stappen; de uiteindelijke geserialiseerde lengte wordt altijd gemeten. Te groot geeft PLAN_TOO_LARGE, nooit truncatie. Voor UTC-expiry is een geldige klokhandshake nodig; zonder betrouwbare tijd geen nieuwe hardware-START.

11.2 Events, fouten en eindevidence

De server toont wat de robot bevestigt, niet wat de planner verwacht.

Eventveld	Betekenis
event_id / event_seq	Stabiel uniek ID per gebeurtenis, monotone sequence per sessie; retry verandert het ID niet.
mission_id / revision / step_id	Exacte opdracht en stap waarop dit event betrekking heeft.
map_version / plan_hash	Welke topologie en uitvoerroute werkelijk actief waren.
source / robot_id / session_id	simulation of hardware en herkomst; nooit vermengen.
event_kind / state	Bijvoorbeeld COMMAND_APPLIED, NODE_OBSERVED, SERVICE_COMPLETED, SEARCHING, RESULT.
pose / observation_seq	Laatste node/edge, aankomstrichting en UNKNOWN/ESTIMATED/ANCHORED.
time_ms / time_utc	Monotone volgorde plus optionele gevalideerde UTC; geen aftrekken tussen verschillende klokken.
payload	Evidence, foutcode, bezocht-prefix, actuele stap en reden van herstel; geen wachtwoorden.

Kritieke eventsoorten en codes

COMMAND_ACCEPTED, COMMAND_DEFERRED, COMMAND_APPLIED, SERVICE_COMPLETED en RESULT gaan via de kritieke outbox en blijven beschikbaar tot event_ack. Telemetrie mag worden samengevoegd. Foutcodes zijn onder andere INVALID_SCHEMA, MAP_MISMATCH, STALE_REVISION, WRONG_BOUNDARY, DUPLICATE_COMMAND, EXPIRED_COMMAND, SERVICE_POINT_UNVERIFIED, POSE_UNCERTAIN, UNREACHABLE en PLAN_TOO_LARGE.

Succescriterium

RobotAgent mag COMPLETED uitsluitend afgeven nadat de servicehistorie in de gewenste volgorde is vastgelegd, de verblijftijden zijn voltooid, de eindpose het juiste anker bevestigt en RouteExecutor END heeft uitgevoerd. Resultaat bevat bezocht-prefix, eindanker, revisie en plan_hash. Bij reset voor ondubbelzinnige eindevidence wordt de oude missie niet alsnog succesvol verklaard.

Geen fysiek exactly-once-bewijs na stroomverlies

Idempotente software voorkomt veel dubbele opdrachten. Een stroomuitval kan echter plaatsvinden tussen een fysieke handeling en het duurzaam opslaan daarvan. Na boot zijn motoren uit en is pose UNKNOWN; operator-/ankerreconciliatie is dan nodig, geen blind automatisch hervatten.

12 Nieuwe instructies tijdens een rit

Volgorde, atomair toepassen en offlinebeleid

Stap	Server	RobotAgent
1	Maak REPLACE rev2 vanaf benoemd toekomstig anker.	Blijft rev1 uitvoeren; ontvangt en valideert rev2.
2	Ontvang DEFERRED en toon pending route gestippeld.	Bewaart maximaal één volgende revisie; lagere revisie afwijzen.
3	Wacht op boundary-event.	Maakt huidige manoeuvre af; bevestigt valid_from_node met vereiste anker-evidence.
4	Verwerkt APPLIED rev2; past UI-snapshot aan.	Installeert volledige rev2 atomair; uitgevoerde serviceprefix blijft gelijk.
5	Stuurt event_ack; bewaart auditspoor.	Voert rev2 uit, herhaalt kritieke events met hetzelfde ID tot applicatiebevestiging.

Een gewone gelijkvormige T is niet automatisch een betrouwbaar wijzigingsanker. Waar ankers ontbreken, kan de wijziging pas op een later gevalideerd servicepunt of na expliciete operatorbevestiging worden toegepast. De UI toont deze beperking; er wordt niet stil vanaf een geschatte kruising herpland.

Komt rev3 binnen terwijl rev2 nog pending is, dan vervangt rev3 die alleen na volledige validatie van dezelfde grens en serviceprefix. Rev2 krijgt SUPERSEDED; de actieve rev1 verandert pas bij APPLIED. Een ongeldige hogere revisie verdringt geen geldig pending plan.

Prioriteit en eigenaarschap

Obstakel/handstop heeft voorrang op bewegen. PAUSE en CANCEL worden op hun ID bevestigd; CANCEL geeft nooit COMPLETED. RobotAgent bezit de actieve revisie; de server bewaart gewenste en geobserveerde toestand afzonderlijk. Een MQTT-callback mag geen lijst onder RouteExecutor vervangen.

PAUSED bewaart de onderbroken toestand en planindex. Een obstakel dat verdwijnt heft een gebruikerspauze niet op. Handmatig CONFIRM_ANCHOR is alleen toegestaan met geautoriseerde operator en stilstaande robot in IDLE, PAUSED of SERVICE; tijdens bewegen wordt het afgewezen. Automatische markerwaarnemingen hebben hun eigen gevalideerde sensorpad.

Bij netwerkverlies

Default voorstel: voer het reeds volledig geaccepteerde plan alleen verder uit als het offlinebeleid dat toestaat en de plaatsinschatting bruikbaar blijft. Nieuwe plannen worden niet verzonnen. Kritieke gebeurtenissen blijven in outbox/checkpoint en worden bij reconnect opnieuw aangeboden. Een nieuwe hardwareboot vereist een nieuwe sessiehandshake en beginanker. Een niet-gesynchroniseerde UTC-klok accepteert geen nieuwe tijdgebonden opdracht.

Het servicebeleid legt vast of een obstakelpauze meetelt in de verblijftijd. Voor v1 telt echte verstreken stationstilstand mee zolang de positie geldig blijft; tijd buiten het station of met onzekere plaats telt niet. Zo is een 5-secondenservice toetsbaar in plaats van afhankelijk van sleep-aanroepen.

13 Simulatiemodi en kalibratiegrenzen

Van logica naar gemeten sensorinput

Niveau	Wat het model doet	Wat het niet bewijst
S1 - topologie	Deterministische node-/edgepassages en routeacties op FakeClock; ideale synthetische input.	Geen echte sensorrespons, motorvermogen, remweg of draaigeometrie.
S2 - sensorreplay	Opgenomen vijfbitsframes en opdrachten herspelen door dezelfde detector/agent.	Geen volledige reactie van een nieuwe motoractie op de omgeving; input is opgenomen.
S3 - eenvoudige kinematica	Gesimuleerde differentiele aandrijving, vijf sensorposities op gemeten lijngeometrie, ruis/drift/voedingprofielen.	Geen gevalideerde natuurgetrouwe dynamica zonder voertuigmetingen; geen garantie tegen slip/lichtinvloed.

Ground truth blijft buiten de besturing

SimHardware mag true_pose kennen om frames te genereren en een onafhankelijk orakel te voeden. RobotAgent ontvangt uitsluitend SensorFrame, commands en de eigen tijdbron. Een verborgen true_node_id mag niet als extra ingang worden gebruikt om een gemist kruispunt alsnog goed te tellen. In de UI worden waarheidspose en geschatte pose duidelijk apart weergegeven.

Tijd en reproduceerbaarheid

FakeClock.advance bepaalt de simulatiestap; browser-FPS bepaalt alleen tekenen. Een scenario bewaart seed, kaartversie, kalibratie, ruismodel, inputopdrachten en verwacht resultaat. Replay vereist ruwe frames en opdrachten. Alleen een tekstlog met DRAAI_KLAAR is onvoldoende om detector- of PID-gedrag opnieuw uit te voeren.

Beperking van de huidige kaart

De coordinaten in het ontwerpartifact zijn schematische eenheden. S3 mag niet doen alsof dit millimeters zijn. Meet eerst schaal en sensor-/voertuiggeometrie en maak een nieuwe goedgekeurde map_version met lengths/geometry in mm.

14 Bediening, API en journaling

Functioneel ontwerp van het nieuwe simulatieproject

Schermindeling

Links de volledige SVG-kaart met vaste IDs, geselecteerde bestemmingsvolgorde en actief plan. Een richtingpijl toont de geschatte inkomende edge; kleur/label toont poseconfidence. Rechts staan missie-ID, revisie, pending wijziging, bezocht-prefix en resultaat. Onderaan een filterbare eventtijdlijn met export van run-evidence.

De modus SIMULATIE of HARDWARE is altijd zichtbaar. Hardware-activering vergt een bewuste, geautoriseerde handeling; een herladen browser start nooit een robot. Onbekende service-/kalibratiepunten krijgen een waarschuwing in het ontwerpscherm en blokkeren hardware-START.

API	Doel
GET /api/map	Actieve kaart + versie en verificatiestatus.
POST /api/missions	Bestemmingen, dwell en eindpunt valideren; planvoorvertoning/START.
POST /api/missions/{id}/revisions	REPLACE met verwachte vorige revisie en toepassingsgrens.
POST /api/missions/{id}/control	PAUSE/RESUME/CANCEL; geautoriseerde ankerbevestiging apart auditen.
POST /api/simulations	Scenario-ID, seed en mode; start uitsluitend een sim-run.
GET /api/state en /ws/events	Snapshot en geordende live events; resync bij gat in sequence. [B07]
GET /api/runs/{id}/export	Config, plan, events en waar beschikbaar ruwe frames; secrets verwijderd.

Journalinhoud en bewaarbeleid

Elke run heeft run_id, source, software/hash, map_version, schema_version, calibration_id, inputopdrachten, seeds en events. Kritieke robotresultaten blijven bewaard tot application-ACK en checkpoint. Grote ruwe opnamen staan buiten Git; kleine golden traces zijn geanonimiseerd. Retentie en maximale schijfruimte worden bij oplevering vastgelegd, niet onbeperkt laten groeien.

15.1 Meetbare acceptatiecriteria

Voorgestelde vrijgave-eisen; geen claim dat de toekomstige implementatie hier al aan voldoet.

ID / eisen	Acceptatiecriterium	Scenario's
AC01 REQ01/14	Kaart heeft 27 unieke nodes, 32 unieke bidirectionele edges, 6 eindpunten, 1 component, 6 cycli en 112 legale lokale overgangen. Iedere getekende poort is precies eenmaal verbonden.	T01-T03
AC02 REQ01/08/20	Alle 5 attracties en het depot hebben een expliciet service-/ankerbeleid. Hardware-start wordt geweigerd zolang een gebruikt servicepunt of noodzakelijke kalibratie niet gevalideerd is.	T04/T43
AC03 REQ02/03	Alle 702 verschillende geordende nodeparen en 1728 combinaties van gerichte aankomsttoestand met doel zijn structureel bereikbaar. Elk pad gebruikt alleen toegestane edges/overgangen.	T05-T06
AC04 REQ02/03/17	Alle 325 geordende selecties van 1 t/m 5 verschillende attracties, gevolgd door depot, leveren een geldig plan op. Hopkosten komen overeen met een onafhankelijke BFS-orakelcontrole.	T07-T10
AC05 REQ05	Elke fysiek/gesimuleerd bevestigde passage veroorzaakt precies één node-event. Een bevestigde uitgang wordt hergebruikt; een tweede vrijgavewachttijd mag het volgende punt niet verbergen.	T11-T13
AC06 REQ05/12	SensorFrame heeft 5 bits, zwart=1, monotone sample_seq. Ongeldige/stale frames leiden tot expliciet herstel, nooit tot een succesvolle afslag. PWM blijft binnen [-1,1].	T14-T15
AC07 REQ13	Voorgestelde hardwareprestatie: over 60 s met 10 MQTT-berichten/s is p99 van framebeschikbaarheid tot motorupdate <=10 ms en maximum <=20 ms. Tick-doel 5 ms; geen garantie zonder boardmeting.	T16/T44
AC08 REQ19	Dwell=5000 ms wordt in FakeClock exact uitgevoerd en op hardware binnen +/-100 ms. Bij dwell=0 en tijdens metingen geen gepland set_motors(0,0); obstakel, handstop en fault zijn expliciete uitzonderingen.	T17-T18

15.2 Meetbare acceptatiecriteria

Voorgestelde vrijgave-eisen; geen claim dat de toekomstige implementatie hier al aan voldoet.

ID / eisen	Acceptatiecriterium	Scenario's
AC09 REQ06/14	Elke opdracht voldoet aan het contract en verwijst naar de actieve map_version, sessie, mission_id en revisie. Een ongeldige opdracht installeert geen enkel plan en geeft een reden terug.	T19-T20
AC10 REQ06/18	Tien bezorgingen met hetzelfde command_id veroorzaken één toepassing. Kritieke event-ID blijft gelijk bij retry; server verwerkt één resultaat. Geen claim van fysiek exactly-once na stroomverlies.	T21-T22
AC11 REQ06/09	In het afgesproken LAN-profiel is p95 van verzenden tot application-ACK <=500 ms. State/snapshot wordt ten minste elke seconde aangeboden; UI markeert data na 3 gemiste intervallen als verouderd.	T23/T40
AC12 REQ07	REPLACE met hogere revisie wordt tijdens rijden ontvangen, maar alleen op valid_from_node met passend anker en onveranderde uitgevoerde prefix toegepast. Midden in TURN geen planwissel.	T24-T27
AC13 REQ06/16/18	Verlopen opdracht, oude sessie, oude revisie en een herhaalde retained START worden afgewezen of als duplicaat bevestigd; nooit opnieuw rijden na reconnect alleen door oude berichten.	T20/T28-T29
AC14 REQ08	Bij een gemist/dubbel punt of niet-passende waarneming wordt pose UNKNOWN/ESTIMATED en missie UNCERTAIN. Geen bevestigde aankomst/resultaat zonder vereiste anker-/service-evidence.	T12/T30-T31
AC15 REQ15	COMPLETED vereist de geordende servicehistorie, alle dwell-acties, het juiste eindanker en een uitgevoerde END. Een routeteller of percentage 100 alleen is onvoldoende.	T32-T34
AC16 REQ11	Namespaces sim en hw, source en mode worden aan beide grenzen gecontroleerd. Een sim-START op een hardwarekanaal veroorzaakt 0 motorcommando's.	T35/T42

15.3 Meetbare acceptatiecriteria

Voorgestelde vrijgave-eisen; geen claim dat de toekomstige implementatie hier al aan voldoet.

ID / eisen	Acceptatiecriterium	Scenario's
AC17 REQ10	Herhaling van dezelfde ruwe frames, opdrachten, config en seed geeft dezelfde stappen, manoeuvres, bewijsbesluiten en eventvolgorde. Alleen expliciet niet-deterministische UTC/transportvelden worden genormaliseerd.	T36-T37
AC18 REQ10/13	Na 30 min referentierun blijven queues/ringbuffers begrensd. Telemetrie mag worden samengevoegd; kritieke events mogen niet stilzwijgend verdwijnen. Volle kritieke queue leidt tot expliciete degraded/fault-afhandeling.	T38-T39
AC19 REQ05/10/20	Stressruns gebruiken minimaal 3 seeds per profiel en synthetische snelheidsfactoren 0.8/1.0/1.3, plus korte aftakkingen, ruis en drift. Deze factoren zijn testwaarden, geen gemeten accukarakteristiek.	T13/T37/T45
AC20 REQ18	Disconnect voert alleen het reeds geaccepteerde plan uit volgens vastgelegd offlinebeleid. Reset zet motoren uit en pose UNKNOWN; geen automatische hervatting voor nieuw sessiehandshake en ankerbevestiging.	T28-T29/T46
AC21 REQ16	Onbevoegde client kan geen hw-command publiceren. Geen geheimen in repository, journal of browserbundle; buiten een geisoleerd testnetwerk wordt TLS met certificaatcontrole gebruikt.	T41-T42
AC22 REQ17/20	Fysieke vrijgave: 10 opeenvolgende correcte ritten per afgesproken referentiemissie op elk van 3 gemeten voedings-/lastprofielen. Geen verkeerde attractie, gemist bezoek of vals COMPLETED; ruwe evidence en timingmeting bewaard.	T43-T45/T48
AC23 REQ04/12/14	De kern importeert en doorloopt contracttests onder CPython en de vastgelegde MicroPython-versie. Kaart-/configwijziging tijdens een missie wordt geweigerd; nieuwe versie via gecontroleerde vrijgave.	T20/T47
AC24 REQ09	UI toont bron, software/kaart/revisie, poseconfidence en afgewezen/pending opdrachten. Reload/reconnect haalt een snapshot op; ontbrekende events worden niet als actuele routeanimatie ingevuld.	T23/T40/T48

16 Teststrategie en orakels

Geen route claimen op basis van alleen een groen animatiepijltje

Laag	Uitvoering	Onafhankelijke controle
Unit	Kaart, TurnRule, planner, schema, detector, tijd, PID.	BFS-orakel naast Dijkstra; vastgelegde golden vectors.
Scenario	RobotAgent + SimHardware + FakeClock.	Verborgen waarheid versus geschatte pose, verwachte services en motortrace.
Replay	Gemeten frames/commands met vaste tijdlijn.	Gelijke genormaliseerde events en expliciete afwijkingen.
Integratie	Echte testbroker, API/WebSocket en reconnects.	Command/event-ID, acknowledgements, topic-ACL en snapshots.
Hardwarebench	Doelboard met meetbare frame-/motorupdate-timestamps.	Timingmeting, GPIO/polariteit, respons en begrensde queues.
Baanacceptatie	Afgesproken missies en voedings-/lastprofielen.	Waarnemer/video/ankerevidence plus volledige run-export.

Testvoorwaarden

- Leg software-, kaart-, schema-, board- en kalibratieversie vast voor iedere run.
- Gebruik vaste seeds en een virtuele klok voor softwaretests; geen sleeps voor reproduceerbare scenario's.
- Definieer voor elke test de input, het meetpunt, het verwachte resultaat en de herhaalbaarheid.
- Vergelijk werkelijke motor-/servicegebeurtenissen met een orakel, niet uitsluitend de interne stapindex.
- Voer fysieke tests eerst per overgang/servicepunt uit en daarna als volledige missie.
- Elke mislukte fysieke terugweg wordt als ruwe opname een regressietest; overschrijf het bewijs niet.

Resultaatstatus in dit rapport

De 48 scenario's zijn een testplan voor de nieuwe applicatie. Ze zijn nog niet als systeemtests uitgevoerd. Alleen de expliciete document-/kaartcontroles in hoofdstuk 22 zijn tijdens het opstellen uitgevoerd.

16.1 Testscenario's 01-08

Elk scenario heeft stimulus, verwacht resultaat en testlaag; uitvoeringsstatus: gepland.

ID / scenario	Invoer en procedure	Verwacht resultaat / orakel	Laag
T01 Kaartinventaris	Laad park_v1.json; vergelijk elk genummerd punt en elke lijn met foto en node-/edgecatalogus.	27 nodes, 32 edges; servicepunten nog als niet-gevalideerd gemarkeerd.	unit + review
T02 Defecte poort of ID	Fixtures: dubbele node-ID, ontbrekende edge-node, dubbel gebruikte poort, lege stationsverwijzing.	Validator geeft stabiele foutcode; planner start niet.	unit
T03 Connectiviteit en overgangen	Controleer componenten, cycli, graden en alle combinaties van lokale poorten.	1 component, 6 cycli, 112 legale overgangen; geen U-bocht op gewone junction.	unit
T04 Onbekend servicepunt	Vraag hardwaremissie naar station met service_node_id=null of hardware_validated=false.	REJECTED: SERVICE_POINT_UNVERIFIED; 0 motorcommando's. Sim-run blijft herkenbaar simulatie.	integration
T05 Alle nodeparen	Plan alle 27*26 verschillende bron-doelparen in hopmodus; controleer edges en adjacency onafhankelijk.	702 geldige paden; zelfde minimale kosten als BFS.	unit
T06 Alle aankomstrichtingen	Voor elk van 64 gerichte edge-eindtoestanden plan naar elk van 27 doelen.	1728 geldige resultaten; lokale poorten bepalen correcte links/rechts.	unit
T07 Alle attractievolgorden	Genereer permutaties met lengte 1 t/m 5 uit de vijf attracties; start/eind depot.	325 legale geordende missies; elk gevraagd servicepunt exact eenmaal.	simulation
T08 Twee bekende attracties	Missie WILDWATER, FREEFALL, depot; dwell=5000 ms; leg vertrekport expliciet vast.	Volgorde en eindpunt kloppen; terugweg passeert Wildwater zonder tweede service.	simulation

16.2 Testscenario's 09-16

Elk scenario heeft stimulus, verwacht resultaat en testlaag; uitvoeringsstatus: gepland.

ID / scenario	Invoer en procedure	Verwacht resultaat / orakel	Laag
T09 Vier attracties doorrijden	COASTER, FREEFALL, WILDWATER, ARCADE, depot met dwell=0.	Geen servicewachttijden; COMPLETED uitsluitend na eindanker depot.	simulation
T10 Afgesloten verbinding	Zet E09 disabled; plan opnieuw. Herhaal met een fixture die een doel volledig isoleert.	Alternatief pad indien bereikbaar; anders UNREACHABLE, geen leeg succesplan.	unit
T11 Een passage, een telling	Voer gemeten/synthetische 00100 -> 00111 -> 00100 in met ruis tijdens breed punt.	Een punt-event na exitbewijs; geen dubbel tellen.	replay
T12 Eerste-rechtsregressie	Synthetische observaties afgeleid uit campuslog: eerst alleen rechts, daarna links+rechtdoor; eerste actie rechts.	Eerste rechts verbruikt de geplande stap; de volgende opdracht is rechtdoor. Geen claim van een complete ruwe sensoropname.	synthetische regressie
T13 Korte gap na vertrek	Na bevestigde uitgang direct eerste T, daarna twee rechter zijtakken; varieer gap en breedduur.	Eerste T voert rechts uit; zijtakken rechtdoor. Geen tweede vrijgavewachttijd.	replay
T14 Ongeldige/stale bits	Injecteer 000111, ongeldige tekens, herhaalde sample_seq en te oude timestamp.	Geen geldig routepunt; expliciete sensor-/stale-frame-afhandeling.	unit
T15 PID en motorbegrenzing	Reeks positieve/negatieve fouten met variabel dt; integraalbegrenzing en reset na manoeuvre.	Links/rechts teken correct; eindige outputs [-1,1]; dt=0 afgewezen.	unit
T16 Blokkerende netwerkadapter	Injecteer trage publish/read, loggingdruk en 10 berichten/s; meet control-tick-latentie.	I/O staat buiten kritieke tick; afgesproken p99/max gehaald of hardwarevrijgave geblokkeerd.	bench

16.3 Testscenario's 17-24

Elk scenario heeft stimulus, verwacht resultaat en testlaag; uitvoeringsstatus: gepland.

ID / scenario	Invoer en procedure	Verwacht resultaat / orakel	Laag
T17 Halte van vijf seconden	Service op FakeClock; advance tot 4999 en 5000 ms. Herhaal met obstakel tijdens service.	Geen vroeg vertrek; dwell-beleid voor obstakelpauze expliciet en reproduceerbaar.	unit + hardware
T18 Doorrit zonder meetstop	Alle stappen van vier-attractierit met dwell=0; registreer alle set_motors-aanroepen.	Geen gepland (0,0) tussen START en END; nood-/obstakelstop apart geclassificeerd.	simulation
T19 Geldige START	Stuur volledige envelope plus plan met correcte sessie, hash, kaart en beginanker.	RECEIVED, ACCEPTED, APPLIED in juiste volgorde; robot uitvoerder actief.	MQTT integration
T20 Contract-/versiefouten	Fixtures: onbekende enum, ongeldige range, extra veld, hashfout, kaartmismatch en verlopen opdracht.	REJECTED met veld/foutcode; actieve missie en motorsituatie blijven consistent.	integration
T21 Dubbele START	Publiceer 10x hetzelfde command_id met QoS1/retry. Herhaal ook dezelfde HTTP Idempotency-Key voor/na serverrestart.	Een missie-installatie; zelfde key/body geeft dezelfde missie, andere body met dezelfde key een conflict.	MQTT + API integration
T22 Dubbel resultaat	Verlies application-result-ACK; laat agent hetzelfde event_id opnieuw zenden.	UI/server tellen resultaat eenmaal; agent stopt retry na applicatie-ACK.	MQTT integration
T23 Live status	Start missie en meet send->ACK en status->UI onder afgesproken LAN-belasting.	p95 ACK <=500 ms; snapshot/telemetrie <=1 s aangeboden; source/revisie zichtbaar.	integration
T24 Wijziging tijdens FOLLOW	REPLACE rev2 op rev1 vanaf volgend anker, zelfde prefix. Variant: geldige rev3 arriveert terwijl rev2 nog pending is.	DEFERRED tot grens; dan hoogste geldige revisie APPLIED. In variant krijgt rev2 SUPERSEDED; geen terugval.	simulation + MQTT

16.4 Testscenario's 25-32

Elk scenario heeft stimulus, verwacht resultaat en testlaag; uitvoeringsstatus: gepland.

ID / scenario	Invoer en procedure	Verwacht resultaat / orakel	Laag
T25 Wijziging tijdens TURN	Lever rev2 halverwege een draai; laat vervolgens de volgende grens bevestigen.	Huidige manoeuvre blijft intact; wijziging pas daarna atomair toegepast.	simulation
T26 Oude revisie/ander vertrekpunt	Stuur rev1 na rev2 en rev3 met valid_from_node die niet de afgesproken grens is.	STALE_REVISION of WRONG_BOUNDARY; geen stille herplanning vanaf fictieve positie.	integration
T27 Annuleren/pauzeren	PAUSE, RESUME en CANCEL op benoemde mission_id; dubbel bezorgen en obstakelclearance tijdens gebruikerspauze.	PAUSED bewaart stap/resume_state; clear heft gebruikerspauze niet op. CANCEL is geen COMPLETED.	simulation
T28 Offline en retained replay	Verbreek broker tijdens rijden; reconnect met oude retained START en nieuwe sessie.	Alleen geaccepteerd plan volgens offlinebeleid; oude START veroorzaakt geen tweede rit.	MQTT integration
T29 Reset tijdens missie	Reset na een bevestigd bezoek, voor het volgende punt; herstel lokaal checkpoint en brokerverbinding.	Motoren uit bij boot; pose UNKNOWN; geen automatische fysieke hervatting.	hardware bench
T30 Kruispunt gemist/dubbel	Laat een breed patroon weg of dupliceer een punt-event; verwachte uitgangen wijken af.	Missie UNCERTAIN; geen fictief anker, geen succes op basis van teller.	fault simulation
T31 Lijnverlies en herstel	Injecteer 00000 gedurende lange tijd, daarna lijn plus geldig anker; timeoutbeleid=null.	SEARCH blijft beschikbaar; actieve stap niet verbruikt door tijd; nieuwe plannen wachten op anker.	replay
T32 Vals depotvlak	Injecteer lang zwart vlak bij Wildwater, ook als alle route-instructies verbruikt lijken.	Zonder correct depotanker geen COMPLETED.	fault simulation

16.5 Testscenario's 33-40

Elk scenario heeft stimulus, verwacht resultaat en testlaag; uitvoeringsstatus: gepland.

ID / scenario	Invoer en procedure	Verwacht resultaat / orakel	Laag
T33 Een bezoek ontbreekt	Simuleer eindanker depot met een ontbrekende service-evidence of onvoltooide dwell.	Geen COMPLETED; reden MISSING_SERVICE of DWELL_INCOMPLETE.	unit
T34 Correcte afsluiting	Alle bezoeken, dwells en eindanker bevestigd; voer END uit en herbezorg resultaat.	Een unieke completion-event; motoren uit; UI missie afgerond.	integration
T35 Simbericht op hardware	Stuur een plan/event met source=simulation of sim-topic naar hw-ingang.	ACL/adapter/agent weigeren; 0 fysieke motoropdrachten.	security integration
T36 Exacte replay	Bewaar volledige ruwe frames, commando's en config; voer tweemaal uit met FakeClock.	Gelijke genormaliseerde eventtrace, stappen en bewijsbesluiten.	replay
T37 Seeds en snelheidsstress	Drie seeds per synthetisch profiel 0.8/1.0/1.3; identieke map/mission/config.	Elke run reproduceerbaar; afwijkingen zichtbaar als failures/uncertainty, nooit stil succes.	fault simulation
T38 Lange run en wachtrijen	30 min scenario met hoge telemetrie, disconnects en terugkerende routeopdrachten.	Begrensd geheugengebruik; kritieke events bevestigd of expliciet in degraded toestand.	soak
T39 Buffers/cache vol	Vul telemetrie, kritieke eventqueue en dedupcache; simuleer trage opslag.	Telemetriecoalescing zichtbaar; geen stil verlies. Nog geldige dedup-records niet evicten; nieuwe opdracht zo nodig afwijzen.	bench
T40 Browser reconnect/stale data	Sluit WebSocket, sla event_seq over, herlaad pagina en laat snapshot oud worden.	Resync aangevraagd; UI toont stale/UNKNOWN en springt niet naar verzonnen positie.	UI integration

16.6 Testscenario's 41-48

Elk scenario heeft stimulus, verwacht resultaat en testlaag; uitvoeringsstatus: gepland.

ID / scenario	Invoer en procedure	Verwacht resultaat / orakel	Laag
T41 Onbevoegde opdracht	Client zonder hw-publishrecht probeert START, REPLACE en handmatige ankerbevestiging.	Broker/API weigeren; auditlog zonder wachtwoorden.	security
T42 TLS/secrets/modes	Ongeldig certificaat, verkeerde brokerconfig, bundle-inspectie en sim/hw-schakeltest.	Verbinding geweigerd bij certificaatfout; geen geheimen in browser/repo/journal.	security
T43 Servicepunt en pinnen	Meet GPIO/motorpolariteit, vijf sensoren, markers, afstand tot wielas en stationstoppositie.	Ondertekend survey-/kalibratierecord; alleen gemeten servicebeleid vrijgegeven.	hardware commissioning
T44 Timing op echte ESP32	60 s per belastingsprofiel, met sensortimestamps en motorupdate-markering/meetapparatuur.	p99 <=10 ms, max <=20 ms; board-, firmware- en MQTT-clientversie vastgelegd.	hardware timing
T45 Voeding/last/bocht	Test gemeten volle/middelste/lagere toegestane voeding en belasting; dezelfde referentiemissies.	Geen gemist doel of vals succes; route- en timingbewijs opgeslagen; geen claim dat PWM snelheid meet.	hardware acceptance
T46 Verbinding-/stroomherstel	Brokeruitval, wifi-uitval en reset afzonderlijk tijdens service en transit.	Gedrag volgt offlinebeleid; nieuwe sessie vereist; status niet ten onrechte succesvol.	hardware acceptance
T47 MicroPython-contractpariteit	Importeer gedeelde kern op doelboard; voer golden vectors voor kaart, messages, timerwrap en detector uit.	Zelfde contract-/uitvoerbesluiten als CPython; geen ontbrekende desktopimport.	cross-runtime
T48 Eindacceptatie en overdracht	Bediener start referentieritten, wijzigt een rit, controleert resultaat en downloadt evidence.	10 correcte opeenvolgende ritten per missie/profiel; handleiding en herstelprocedure uitvoerbaar.	system acceptance

17 Traceerbaarheid

Iedere eis heeft criteria en toetsbare scenario's.

Eis	Onderwerp	Criteria	Scenario's
REQ01	Eén volledige versie van de kaart	AC01, AC02	T01-T03 / T04/T43
REQ02	Variabele bestemmingen	AC03, AC04	T05-T06 / T07-T10
REQ03	Rijrichting en terugweg	AC03, AC04	T05-T06 / T07-T10
REQ04	Eén uitvoeringsmodel	AC23	T20/T47
REQ05	Betrouwbare waarneming	AC05, AC06, AC19	T11-T13 / T14-T15 / T13/T37/T45
REQ06	MQTT-opdrachtcyclus	AC09, AC10, AC11, AC13	T19-T20 / T21-T22 / T23/T40 / T20/T28-T29
REQ07	Wijziging tijdens rijden	AC12	T24-T27
REQ08	Geen fictieve positie	AC02, AC14	T04/T43 / T12/T30-T31
REQ09	Bediening en observatie	AC11, AC24	T23/T40 / T23/T40/T48
REQ10	Reproduceerbaarheid	AC17, AC18, AC19	T36-T37 / T38-T39 / T13/T37/T45
REQ11	Scheiding sim en hardware	AC16	T35/T42
REQ12	MicroPython-overdraagbaarheid	AC06, AC23	T14-T15 / T20/T47
REQ13	Reactietijd	AC07, AC18	T16/T44 / T38-T39
REQ14	Versies en configuratie	AC01, AC09, AC23	T01-T03 / T19-T20 / T20/T47
REQ15	Aantoonbaar succes	AC15	T32-T34
REQ16	Beveiligde toegang	AC13, AC21	T20/T28-T29 / T41-T42
REQ17	Meetbare kwaliteit	AC04, AC22	T07-T10 / T43-T45/T48
REQ18	Verbindings-/herstartgedrag	AC10, AC13, AC20	T21-T22 / T20/T28-T29 / T28-T29/T46
REQ19	Dynamische verblijftijden	AC08	T17-T18
REQ20	Bewijs en vrijgave	AC02, AC19, AC22	T04/T43 / T13/T37/T45 / T43-T45/T48

De volledige criteria/scenariocatalogus is ook machineleesbaar als PDF-bijlage opgenomen. De links tussen IDs zijn gecontroleerd. Een gedeeltelijk uitgevoerde kaartcontrole betekent niet automatisch dat een volledig acceptatiecriterium is geslaagd.

18 Implementatie- en migratieplan

Gefaseerd bouwen zonder de bestaande robotproeven te overschrijven

Fase	Werk	Uitgangscriterium
P0 - survey	Kaart en IDs reviewen; servicepunten, markers, pinnen en versies inventariseren.	Topologie akkoord; ontbrekende meetwaarden expliciet geregistreerd.
P1 - domein	Model, GraphValidator, TurnRule en planner implementeren; JSON als bron.	T01-T10 slagen op ideale topologie; SVG uit dezelfde bron.
P2 - contract	Schemas, command/event-ID, revisies en MQTT-proefbroker bouwen.	T19-T29; geen robotmotoren verbonden met sim-kanalen.
P3 - uitvoerder	Korte tick-state-machine, detector, pose en servicebeleid uit bestaande code losmaken.	Unit/replay; geen lange sleep of nested volledige zoeklus.
P4 - simulator/UI	InMemoryChannel, FakeClock, scenario's, SVG, journal en exports.	T11-T18, T30-T40; modus/waarheid zichtbaar gescheiden.
P5 - firmwareadapter	MicroPython-imports en golden vectors; MQTT-client en RAM/timing benchtesten.	T43/T44/T47; alleen goedgekeurde servicepunten activeren.
P6 - baanacceptatie	Losse segmenten, services en terugwegen, daarna referentieritten en wijziging onderweg.	AC22 met bewijs; operatorhandleiding en rollback gecontroleerd.

Herbruikbaar uit de huidige code

Pin-/motorpolariteit als gecontroleerde configuratie, sensor-normalisatie naar vijf bits, PID als basis voor kalibratie, diagnosebehoeften en concrete regressies uit de mislukte ritten. De huidige vaste RITPLAN-lijsten worden testfixtures; ze worden niet een tweede routeplanner naast de kaart.

Overdracht en versies

Maak een nieuw project/repository voor dit ontwerp. Bewaar een herkenbare laatste robotversie en een terugzetprocedure. Pin dependencies en MicroPython-build bij implementatie; documentatie op /latest kan ontwikkelfuncties beschrijven. Credentials via omgeving/config buiten Git. Iedere firmware-uitrol vermeldt kaart-, schema- en calibration_id en heeft een startcontrole met motoren uit.

19 Risico's en beheersing

Risico's gekoppeld aan bewijs, geen losse waarschuwingslijst

ID / niveau	Risico	Beheersing	Tests
RIS01 Hoog	Eén gemist punt verschuift de hele route	PoseEstimator + ankers; mismatch naar UNCERTAIN; geen teller-alleen-succes.	T12/T30/T32
RIS02 Hoog	Servicepunt verkeerd uit foto afgeleid	Station.terminal_node en service_node scheiden; fysieke survey verplicht.	T04/T43
RIS03 Hoog	Volle accu verandert snelheid/draaipositie	Meet voedings-/lastprofielen; tijdvensters kalibreren; PWM niet als snelheid labelen.	T15/T45
RIS04 Hoog	MQTT/logging blokkeert sensoren	Queues en niet-blokkerende tick; concrete firmwareclient benchtesten.	T16/T44
RIS05 Hoog	Nieuwe route midden in draai	Deferred revisie met valid_from_node; agent is eigenaar van toepassing.	T24-T26
RIS06 Hoog	Dubbele/oude opdrachten starten opnieuw	ID/revisie/sessie/expiry, retain=false, applicatie-ACK en idempotentie.	T21/T28/T29
RIS07 Hoog	Simulator lijkt correct door verborgen waarheid	Ground truth alleen naar testorakel; geen true_node_id in agentinput.	T30/T36
RIS08 Hoog	Sim kan echte robot aansturen	Gescheiden topics/credentials, broncontrole en fysieke activeringsstap.	T35/T42
RIS09 Hoog	Na reset onterecht dezelfde fysieke positie	Boot motoren uit; nieuw sessie-ID; opnieuw ankeren voor hervatten.	T29/T46
RIS10 Middel	Oneindig zoeken zonder bruikbare lokalisatie	SEARCH mag zonder timeout; missie blijft UNCERTAIN en wijzigingen wachten op anker.	T31
RIS11 Middel	Bufferverlies maakt diagnose onmogelijk	Kritiek journal met ACK; ruwe ringopname apart; overflow zichtbaar.	T38/T39
RIS12 Hoog	Kaart- of contractversies lopen uiteen	Hashes/version handshake; geen live kaartwissel tijdens missie.	T20/T47

Alle hoge risico's moeten een eigenaar krijgen in het nieuwe project. Geaccepteerde restrisico's worden vastgelegd bij release; onbekende servicepunten, niet-gemeten timing en onzekere positie mogen niet als bewezen correct worden afgevinkt.

20 Ontwerpbeslissingen en alternatieven

Besluiten die consistent moeten blijven tijdens implementatie

ADR	Besluit en reden	Afweging
ADR01	Topologie in JSON, SVG gegenereerd.	Voorkomt twee afwijkende kaarten; foto blijft bewijsbron, geen runtime-routebestand.
ADR02	Dijkstra op aankomsttoestand en lokale poorten.	Iets meer toestand dan node-only; voorkomt verkeerde links/rechts bij gebogen edges.
ADR03	Planner op server, uitvoering op robot.	Voldoende ruimte voor UI/tests; ESP32 blijft autonoom voor reeds geaccepteerd plan.
ADR04	Gedeelde kern met adapters, geen GPIO in domein.	Eenmalige refactor nodig; daarna dezelfde beslissingen onder FakeClock en hardware.
ADR05	Tick-FSM, geen volledige manoeuvre in while/sleep.	Meer expliciete states; continue opdracht-/obstakelverwerking mogelijk.
ADR06	Application-ACK en idempotentie naast MQTT-QoS.	Extra administratie; voorkomt transport-ACK verwarren met fysiek succes.
ADR07	Servicepunt/anker eerst bewijzen.	Extra commissioning; voorkomt een herkenbare T zonder bewijs als attractie/depot labelen.
ADR08	S1, S2 en S3 apart labelen.	Meer zichtbare modelgrenzen; geen ongefundeerde fysische nauwkeurigheid.
ADR09	Browser-SVG + FastAPI, geen zware frontend nodig voor v1.	Eenvoudige bediening/testbaarheid; frameworkmigratie later mogelijk zonder kernwijziging.
ADR10	Geen automatische fysieke hervatting na reset.	Operator-/ankerhandeling vereist; voorkomt opnieuw uitvoeren vanuit een onjuiste pose.

21 Aanvullingen op de oorspronkelijke vraag

Benodigde stappen die naast een kaart en simulator horen

- Een fysiek surveyformulier en afspraken over servicepunten en unieke ankers. Dit is nodig voor betrouwbare aankomst, terugweg en succesmelding.
- Een definitie van locatiezekerheid en herstel na gemiste kruispunten. Kaarttelling alleen is onvoldoende.
- Een formeel MQTT-contract met ID, sessie, revisie, expiry, pending wijziging en application-result-ACK.
- Een niet-blokkerende uitvoeringskern en meetbaar tijd-/geheugenbudget voor de concrete ESP32/client.
- Een duidelijk verschil tussen gesimuleerde waarheid, geschatte positie en fysiek bevestigde ankers.
- Versiebeheer voor kaart, kalibratie, schema, software en dependencies; herhaalbare build en rollback.
- Broker-ACL, geheimenbeheer en onmiskenbare scheiding van simulatie en hardware.
- Ruwe sensoropnamen en een replay-/incidentprocedure; alleen tekstlogs reconstrueren de route niet.
- Een expliciet offline-/resetbeleid en beheer van kritieke eventqueues.
- Een fysieke acceptatiepoort met meerdere voedings-/lastprofielen en onafhankelijke waarneming.

Reviewchecklist voor het team

Bevestig de 27 punten en 32 lijnen; besluit per station terminal versus servicepunt; wijs eigenaren toe voor kaart, firmware, MQTT/server en acceptatie; leg het echte board/client vast; spreek de referentiemissies en meetcriteria af. Pas daarna kan dit ontwerp als implementatiebaseline worden goedgekeurd.

22 Uitgevoerde ontwerpcontroles

Wat werkelijk is nagekeken bij het maken van deze PDF

Controle	Uitkomst	Betekenis
Kaartintegriteit	27 nodes / 32 edges / 112 rules	IDs, unieke poorten, graden en verwijzingen consistent.
Connectiviteit	1 component / 6 cycli	Alle punten structureel verbonden.
Nodeparen	702 paden	Dijkstra-hopkosten gelijk aan onafhankelijke BFS.
Gerichte aankomsttoestanden	1728 paden	Alle 64 edge-eindtoestanden naar 27 doelen bereikbaar.
Attractievolgorden	325 missies	Unieke geordende selecties met retour depot; inkomende edge behouden.
Capaciteit kaartbasis	Max. 49 transit / 55 totaal	Onder voorgestelde 64-stappengrens; bytes blijven per bericht te meten.
UML/registratie	42 klassen / 6 enums / 66 relaties	Klasse-ID, bestand, methode, relatiedoel en catalogus uit dezelfde bron.
Traceerbaarheid	20 eisen / 24 criteria / 48 scenario's	Alle eisen hebben criteria; alle scenarioverwijzingen bestaan.
PDF-kwaliteit	Na genereren visueel en tekstueel gecontroleerd	Controle op afkapping, paginavolgorde, diagrammen en leesbaarheid.

Dit zijn ontwerp-/documentcontroles met een klein controleprogramma, geen uitgevoerde systeemacceptatie van de nieuwe applicatie. De voorgestelde 48 scenario's, MicroPython-adapter, MQTT-belastingtest en fysieke ritten zijn nog uit te voeren. De toekomstige productieplanner krijgt eigen tests; deze controlecode is een onafhankelijk ontwerporakel.

Bronidentificatie

`Foto SHA-256: 6e137e7f6de8af69ea04b4544f28d3be51083c9426dfb45ccfc2fadc69f28cf3`

`Huidige main.py SHA-256: 53ba8333414ffa33338dae33dfc41efed7a1bef31d89092f170f0564a0f03031`

Er zijn geen robotbestanden gewijzigd tijdens het maken van dit rapport. In de PDF zitten als machineleesbare bijlagen: map_design.json, class_model.json, test_catalog.json, sim_profile.json en design_checks.json. De zichtbare tekst en tabellen bevatten de inhoud die voor beoordeling nodig is.

23 Bronnen en begrippen

Primaire technische documentatie, geraadpleegd 3 november 2026

B01 - Eigen bovenaanzicht en gesprek

Door gebruiker aangeleverde foto; interpretatie van topologie, geen gemeten schaal.

Eigen bron: foto/gesprek/workspace. Bestanden en hashes staan in hoofdstuk 22.

B02 - Eigen huidige robotcode

main.py en route_instructies.py; MQTT-/objectdetectiemodules ontbreken in de workspace.

Eigen bron: foto/gesprek/workspace. Bestanden en hashes staan in hoofdstuk 22.

B03 - OASIS MQTT 3.1.1

QoS, retained messages, sessions en Last Will.

https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

B04 - MicroPython machine.PWM

PWM duty en hardwareaansturing.

https://docs.micropython.org/en/latest/library/machine.PWM.html

B05 - MicroPython time

ticks_ms, ticks_diff en wrap-around.

https://docs.micropython.org/en/latest/library/time.html

B06 - Eclipse Paho Python

Clientloop, callbacks en netwerkverwerking op de server.

https://eclipse.dev/paho/files/paho.mqtt.python/html/client.html

B07 - FastAPI WebSockets

Bidirectionele browsertelemetrie via de applicatieserver.

https://fastapi.tiangolo.com/advanced/websockets/

B08 - JSON Schema 2020-12

Schema-identificatie en structurele berichtvalidatie.

https://json-schema.org/draft/2020-12

B09 - Python heapq

Priority queue voor een kleine Dijkstra-implementatie.

https://docs.python.org/3/library/heapq.html

Termen: node = kaartpunt; edge = verbinding; poort = lokale aansluitingskant; anker = uniek plaatsbewijs; pose = plaats/richting met zekerheid; tick = korte besturingsstap; dwell = verblijftijd; replay = input opnieuw uitvoeren; idempotent = herhaling heeft geen tweede actie-effect.

Bijlage A - Volledig klassendiagram

UML: + publiek; gevulde ruit = compositie; stippellijn/driehoek = interface-realisatie. R-nummers verwijzen naar de volledige relatietabel.

A.1a Kaart, stations en missie

UML: + publiek; gevulde ruit = compositie; stippellijn/driehoek = interface-realisatie. R-nummers verwijzen naar de volledige relatietabel.



A.1b Stappen, pose en berichten

UML: + publiek; gevulde ruit = compositie; stippellijn/driehoek = interface-realisatie. R-nummers verwijzen naar de volledige relatietabel.

C08 RouteStep

```
+ step_id: str; index: int
+ from_id: str; to_id: str
+ edge_id: str | None
+ maneuver: Maneuver
+ station_id: str | None
+ dwell_ms: int; evidence: str
```

```
+ requires_anchor() -> bool
```

Relaties: R10, R11

C09 RobotPose

```
+ last_node_id: str | None
+ incoming_edge_id: str | None
+ current_edge_id: str | None
+ confidence: PoseConfidence
+ observation_seq: int
```

```
+ is_anchored() -> bool
+ copy() -> RobotPose
```

C10 SensorFrame

```
+ sample_seq: int; time_ms: int
+ bits: str[5]
+ distance_mm: float | None
+ anchor_id: str | None
```

```
+ validate() -> bool
```

C11 TrackObservation

```
+ observation_seq: int; time_ms: int
+ kind: str
+ pattern: str[5]
+ seen_branches: list[str]
+ anchor_id: str | None
+ stable_ms: int
```

```
+ compatible(step, track_map) -> bool
```

C12 RobotCommand

```
+ schema_version: int; command_id: str
+ robot_id: str; session_id: str; source: str
+ kind: CommandKind
+ mission_id: str; revision: int; map_version: str
+ issued_at: str; expires_at: str
+ valid_from_node: str | None
+ payload: dict
```

```
+ validate_shape() -> list[str]
```

C13 RobotEvent

```
+ schema_version: int; event_id: str; event_seq: int
+ robot_id: str; session_id: str; source: str
+ mission_id: str | None; revision: int
+ map_version: str; plan_hash: str | None
+ event_kind: str; state: ExecutorState
+ step_id: str | None; pose: RobotPose
+ time_ms: int; time_utc: str | None
+ observation_seq: int; payload: dict
```

```
+ to_dict() -> dict
```

R12

Relaties: R12

A.2 Besturingskern

UML: + publiek; gevulde ruit = compositie; stippellijn/driehoek = interface-realisatie. R-nummers verwijzen naar de volledige relatietabel.

C14 GraphValidator

```
+ validate_map(track_map) -> list[str]
+ validate_service_points(track_map, hardware) -> list[str]
```

Relaties: R15

C15 RoutePlanner

```
+ track_map: TrackMap
```

```
+ plan(mission, pose, cost_mode) -> RoutePlan
+ path(start_pose, target_id, cost_mode) -> list[RouteStep]
```

Relaties: R13, R14

C16 CommandValidator

```
+ recent_ids: bounded_set
+ active_revision: int
```

```
+ check(command, pose, map_version, transport_meta) -> str
+ remember(command_id) -> None
```

C17 MissionCoordinator

```
+ active: Mission | None
+ pending: RobotCommand | None
+ visited: list[str]
+ request_ids: bounded_map
```

```
+ submit(mission, pose, request_id) -> RobotCommand
+ replace(mission, boundary) -> RobotCommand
+ on_event(event) -> None
+ result_consistent(event) -> bool
+ restore(events) -> None
```

Relaties: R16, R17, R18, R19, R21, R32, R66

C18 PoseEstimator

```
+ pose: RobotPose
+ expected_step: RouteStep | None
```

```
+ update(observation) -> RobotPose
+ anchor(node_id, anchor_id) -> RobotPose
+ invalidate(reason) -> None
```

Relaties: R30, R31

C19 JunctionDetector

```
+ armed: bool
+ candidate_since_ms: int | None
+ last_observation_seq: int
+ thresholds_ms: dict
```

```
+ update(frame) -> TrackObservation | None
+ confirm_exit(observation) -> None
+ reset() -> None
```

Relaties: R28, R29

C20 LineController

```
+ kp: float; ki: float; kd: float
+ integral: float
+ history: bounded_list
+ base_pwm: float
```

```
+ update(frame, dt_ms) -> tuple[float,float]
+ reset() -> None
```

C21 RouteExecutor

```
+ state: ExecutorState
+ resume_state: ExecutorState | None
+ plan: RoutePlan | None
+ step_index: int
+ pending_events: bounded_queue
```

```
+ tick(now_ms, frame, observation) -> None
+ install(plan) -> None
+ pause(reason) -> None
+ resume() -> None
+ drain_events() -> list[RobotEvent]
```

Relaties: R22, R23, R24, R25, R26, R27, R56

C41 RobotAgent

```
+ mission: Mission | None
+ executor: RouteExecutor
+ pending: RobotCommand | None
+ visited: list[str]
+ last_event_seq: int
```

```
+ tick() -> None
+ accept(command) -> str
+ apply_pending_at_anchor() -> None
+ emit(event) -> None
```

Relaties: R20, R44, R58, R59, R60, R61, R62, R63

A.3 Interfaces en enums

UML: + publiek; gevulde ruit = compositie; stippellijn/driehoek = interface-realisatie. R-nummers verwijzen naar de volledige relatietabel.

C22 Clock <<interface>>

```
+ now_ms() -> int
+ diff_ms(a,b) -> int
```

C23 HardwareIO <<interface>>

```
+ read_frame() -> SensorFrame
+ set_motors(left,right) -> None
+ stop() -> None
```

C24 MessageChannel <<interface>>

```
+ receive(limit) -> list[dict]
+ send(message) -> None
+ connected() -> bool
```

C25 EventSink <<interface>>

```
+ append(event) -> None
+ checkpoint() -> None
```

EN01 NodeKind <<enumeration>>

```
+ DEPOT
+ JUNCTION
+ CORNER
+ TERMINAL
```

EN02 Maneuver <<enumeration>>

```
+ STRAIGHT
+ LEFT
+ RIGHT
+ U_TURN
+ SERVICE
+ END
```

EN03 ExecutorState <<enumeration>>

```
+ IDLE
+ FOLLOW
+ APPROACH
+ TURN
+ SERVICE
+ SEARCH
+ OBSTACLE
+ PAUSED
+ COMPLETE
+ FAULT
```

EN04 PoseConfidence <<enumeration>>

```
+ UNKNOWN
+ ESTIMATED
+ ANCHORED
```

EN05 MissionStatus <<enumeration>>

```
+ ACCEPTED
+ EXECUTING
+ UNCERTAIN
+ COMPLETED
+ FAILED
+ CANCELLED
```

EN06 CommandKind <<enumeration>>

```
+ START
+ REPLACE
+ PAUSE
+ RESUME
+ CANCEL
+ CONFIRM_ANCHOR
```

A.4 Adapters en simulatie

UML: + publiek; gevulde ruit = compositie; stippellijn/driehoek = interface-realisatie. R-nummers verwijzen naar de volledige relatietabel.

C26 SystemClock

```
+ now_ms() -> int
+ diff_ms(a,b) -> int
```

Relaties: R33

C27 FakeClock

```
+ time_ms: int
```

```
+ now_ms() -> int
+ diff_ms(a,b) -> int
+ advance(dt_ms) -> None
```

Relaties: R34

C28 Esp32Clock

```
+ now_ms() -> int
+ diff_ms(a,b) -> int
```

Relaties: R35

C29 Esp32Hardware

```
+ pin_config: dict
+ calibration: dict
```

```
+ read_frame() -> SensorFrame
+ set_motors(left,right) -> None
+ stop() -> None
```

Relaties: R36

C30 MqttGateway

```
+ inbox: bounded_queue
+ outbox: bounded_queue
```

```
+ receive(limit) -> list[dict]
+ send(message) -> None
+ connected() -> bool
```

Relaties: R38

C31 FirmwareMqttChannel

```
+ inbox: bounded_queue
+ outbox: bounded_queue
+ session_id: str
```

```
+ receive(limit) -> list[dict]
+ send(message) -> None
+ connected() -> bool
```

Relaties: R39

C32 SimHardware

```
+ mode: str
+ true_pose: dict
+ motor_command: tuple
+ sensor_script: list
+ seed: int
```

```
+ read_frame() -> SensorFrame
+ set_motors(left,right) -> None
+ stop() -> None
+ advance(dt_ms) -> None
```

Relaties: R37, R45

C33 FaultInjector

```
+ seed: int
+ schedule: list[dict]
```

```
+ transform_frame(frame) -> SensorFrame
+ transform_command(command) -> list[RobotCommand]
```

R45

C34 SimulationEngine

```
+ clock: FakeClock
+ hardware: SimHardware
+ agent: RobotAgent
+ tick_ms: int
```

```
+ step() -> list[RobotEvent]
+ run_until(predicate, max_ticks) -> list[RobotEvent]
+ reset(scenario) -> None
```

Relaties: R41, R42, R43

C35 EventJournal

```
+ run_id: str
+ path: str
+ last_event_seq: int
```

```
+ append(event) -> None
+ checkpoint() -> None
+ read(run_id) -> list[RobotEvent]
```

Relaties: R40

R42

R41

C36 ReplayRunner

```
+ run(recording, config) -> list[RobotEvent]
+ compare(expected,actual) -> list[str]
```

Relaties: R46, R47

R47

R46

C42 InMemoryChannel

```
+ inbox: bounded_queue
+ peer: InMemoryChannel | None
```

```
+ receive(limit) -> list[dict]
+ send(message) -> None
+ connected() -> bool
+ pair() -> tuple[InMemoryChannel, InMemoryChannel]
```

Relaties: R64, R65

R65 1:0..1

A.5 Applicatie en browser

UML: + publiek; gevulde ruit = compositie; stippellijn/driehoek = interface-realisatie. R-nummers verwijzen naar de volledige relatietabel.

C37 ApiService

```
+ planner: RoutePlanner
+ coordinator: MissionCoordinator
+ gateway: MqttGateway
```

```
+ submit_mission(request) -> dict
+ replace_mission(request) -> dict
+ start_simulation(scenario_id) -> dict
+ snapshot() -> dict
+ stream_events() -> iterator
```

Relaties: R48, R49, R50, R57

C38 BrowserClient

```
+ selected_mode: str
+ session_id: str
```

```
+ submit_mission() -> None
+ connect_events() -> None
+ reconnect() -> None
```

Relaties: R51, R52, R53

C39 MapRenderer

```
+ map_version: str
+ svg_root: element
```

```
+ draw_map(map) -> None
+ draw_plan(plan) -> None
+ draw_pose(pose) -> None
```

Relaties: R54

C40 TelemetryStore

```
+ events: bounded_list
+ last_seq: int
+ snapshot: dict
```

```
+ apply(event) -> None
+ resync(snapshot) -> None
+ is_stale(now_ms) -> bool
```

Relaties: R55

Bijlage B.1 - Volledige relatietabel

Cardinaliteiten zijn logische koppelingen; veel domeinverwijzingen worden als IDs geserialiseerd, niet als gedeeld geheugen.

ID	Bron	Doel	Type / cardinaliteit	Betekenis
R01	TrackMap	TrackNode	composition; 1 : 1..*	nodes
R02	TrackMap	TrackEdge	composition; 1 : 1..*	edges
R03	TrackMap	Station	composition; 1 : 1..*	stations
R04	TrackMap	TurnRule	composition; 1 : 1..*	turn_rules
R05	TrackEdge	TrackNode	association; 0..* : 2	twee eindpunten via IDs
R06	Station	TrackNode	association; 0..* : 1..2	terminal en optioneel servicepunt
R07	RoutePlan	RouteStep	composition; 1 : 1..*	steps
R08	RoutePlan	RobotPose	composition; 1 : 1	start_pose snapshot
R09	RoutePlan	Mission	association; 0..* : 1	mission_id / revision
R10	RouteStep	TrackEdge	association; 0..* : 0..1	edge_id; service/einde zonder edge
R11	RouteStep	Station	association; 0..* : 0..1	servicebestemming
R12	RobotEvent	RobotPose	composition; 1 : 1	pose snapshot
R13	RoutePlanner	TrackMap	association; 1 : 1	leest
R14	RoutePlanner	RoutePlan	dependency; 1 : 0..*	maakt
R15	GraphValidator	TrackMap	dependency; 1 : 0..*	controleert
R16	MissionCoordinator	Mission	association; 1 : 0..1	active
R17	MissionCoordinator	RobotCommand	association; 1 : 0..1	pending
R18	MissionCoordinator	CommandValidator	composition; 1 : 1	valideert opdrachten
R19	MissionCoordinator	RoutePlanner	association; 1 : 1	plant/replant
R20	RobotAgent	RouteExecutor	composition; 1 : 1	installeert en uitvoert plan
R21	MissionCoordinator	MessageChannel	association; 1 : 1	verzendt plan, ontvangt events
R22	RouteExecutor	RoutePlan	association; 1 : 0..1	active plan

De A1-overzichtspagina bevat alle 48 UML-elementen en alle 66 relaties. Detailbladen tonen de interne relaties binnen hun selectie. Cross-package-relaties staan volledig op A1 en in deze tabel; enumtypen zijn ook expliciet in de attributen benoemd.

Bijlage B.2 - Volledige relatietabel

Cardinaliteiten zijn logische koppelingen; veel domeinverwijzingen worden als IDs geserialiseerd, niet als gedeeld geheugen.

ID	Bron	Doel	Type / cardinaliteit	Betekenis
R23	RouteExecutor	PoseEstimator	composition; 1 : 1	plaatsinschatting
R24	RouteExecutor	LineController	composition; 1 : 1	motorcorrectie
R25	RouteExecutor	HardwareIO	association; 1 : 1	driver
R26	RouteExecutor	Clock	association; 1 : 1	tijd
R27	RouteExecutor	EventSink	association; 1 : 1	kritieke events
R28	JunctionDetector	SensorFrame	dependency; 1 : 0..*	verwerkt
R29	JunctionDetector	TrackObservation	dependency; 1 : 0..*	maakt
R30	PoseEstimator	TrackObservation	dependency; 1 : 0..*	verwerkt
R31	PoseEstimator	RobotPose	composition; 1 : 1	pose
R32	MissionCoordinator	GraphValidator	association; 1 : 1	kaart/uitvoerbaarheid
R33	SystemClock	Clock	realization; -	implementeert
R34	FakeClock	Clock	realization; -	implementeert
R35	Esp32Clock	Clock	realization; -	implementeert
R36	Esp32Hardware	HardwareIO	realization; -	implementeert
R37	SimHardware	HardwareIO	realization; -	implementeert
R38	MqttGateway	MessageChannel	realization; -	implementeert
R39	FirmwareMqttChannel	MessageChannel	realization; -	implementeert
R40	EventJournal	EventSink	realization; -	implementeert
R41	SimulationEngine	FakeClock	composition; 1 : 1	simulatietijd
R42	SimulationEngine	SimHardware	composition; 1 : 1	gesimuleerde hardware
R43	SimulationEngine	RobotAgent	composition; 1 : 1	dezelfde robotcyclus
R44	RobotAgent	JunctionDetector	composition; 1 : 1	waarnemingen

De A1-overzichtspagina bevat alle 48 UML-elementen en alle 66 relaties. Detailbladen tonen de interne relaties binnen hun selectie. Cross-package-relaties staan volledig op A1 en in deze tabel; enumtypen zijn ook expliciet in de attributen benoemd.

Bijlage B.3 - Volledige relatietabel

Cardinaliteiten zijn logische koppelingen; veel domeinverwijzingen worden als IDs geserialiseerd, niet als gedeeld geheugen.

ID	Bron	Doel	Type / cardinaliteit	Betekenis
R45	SimHardware	FaultInjector	association; 1 : 0..1	foutinjectie
R46	ReplayRunner	SimulationEngine	dependency; 1 : 1	herspeelt frames
R47	ReplayRunner	EventJournal	dependency; 1 : 1	leest
R48	ApiService	MissionCoordinator	association; 1 : 1	opdrachtcyclus
R49	ApiService	MqttGateway	association; 1 : 1	hardwarekoppeling
R50	ApiService	SimulationEngine	association; 1 : 0..1	simulatierun
R51	BrowserClient	ApiService	dependency; 1 : 1	HTTP + WebSocket
R52	BrowserClient	TelemetryStore	composition; 1 : 1	weergavetoestand
R53	BrowserClient	MapRenderer	composition; 1 : 1	kaartweergave
R54	MapRenderer	TelemetryStore	association; 1 : 1	leest snapshot
R55	TelemetryStore	RobotEvent	association; 1 : 0..*	begrensde historie
R56	RouteExecutor	JunctionDetector	association; 1 : 1	bevestigt passage/rearm
R57	ApiService	RoutePlanner	association; 1 : 1	voorvertoning plan
R58	RobotAgent	CommandValidator	composition; 1 : 1	opdrachtvalidatie op robot
R59	RobotAgent	MessageChannel	association; 1 : 1	commando/eventkanaal
R60	RobotAgent	HardwareIO	association; 1 : 1	leest frames
R61	RobotAgent	Clock	association; 1 : 1	ticktijd
R62	RobotAgent	EventSink	association; 1 : 1	journal/checkpoint
R63	RobotAgent	Mission	association; 1 : 0..1	actieve missie
R64	InMemoryChannel	MessageChannel	realization; -	implementeert
R65	InMemoryChannel	InMemoryChannel	association; 1 : 0..1	peer; precies twee kanaaleinden per run
R66	MissionCoordinator	EventSink	association; 1 : 1	duurzaam aanvraag-/resultaatjournal

De A1-overzichtspagina bevat alle 48 UML-elementen en alle 66 relaties. Detailbladen tonen de interne relaties binnen hun selectie. Cross-package-relaties staan volledig op A1 en in deze tabel; enumtypen zijn ook expliciet in de attributen benoemd.